

Wrocław University of Science and Technology
Faculty of Electronics, Photonics and Microsystems

FIELD OF STUDY: Control Engineering and Robotics (AIR)

MASTER THESIS

TITLE OF THESIS:

Comparative studies on walking control
algorithms for the quadruped robot 4BarBot

AUTHOR:

Jakub Delicat

SUPERVISOR:

Janusz Jakubiak PhD,
Department of Cybernetics and Robotics

Contents

1	Introduction	6
1.1	Motivation	6
1.2	Commercial and research quadruped robots	7
1.3	Introduction to 4BarBot	8
1.4	Objectives of the Thesis	9
2	Background and Fundamentals	10
2.1	Gaits	10
2.2	Control algorithms	10
2.2.1	Impedance control	11
2.2.2	Model Predictive Control	12
2.2.3	Reinforcement learning	12
2.2.4	Conclusions	12
2.3	Impedance Controller tuning	12
2.4	Kinematics	13
2.5	Inverse kinematics	14
3	Construction of 4BarBot robot	15
3.1	Body construction	15
3.2	Leg construction	16
3.3	Brushless DC Motors in Legged Robotics	16
3.4	Motor controllers	17
3.5	IMU	18
4	Theory of quadruped trot gait	19
4.1	Gait scheduler	19
4.2	Leg kinematics	23
4.2.1	Computing passive angle q_{p3}	25
4.3	Leg inverse kinematics	26
4.4	Impedance controller	30
5	Implementation	31
5.1	Robot Operating System 2	32
5.2	Implementation of the Gait Scheduler	32
5.3	ROS 2 Control	32
5.4	Quadruped controller	34
5.5	Hardware Interfaces	34
5.6	Visualization	35

Contents	2
6 Impedance Control Analysis	36
6.1 Configuration space	36
6.2 Task space	36
7 Conclusion	41
A Time-domain trajectory following plots	42
Bibliography	54

List of Figures

1.1	Four popular quadruped robots: Spot, ANYmal D, Solo, and HyQ. . . .	8
1.2	CAD model of the 4BarBot	9
2.1	Periodic gait patterns of all four legs: FL - front left, FR - front right, RL - rear left, RR - rear right [FPW ⁺ 24]	11
3.1	Photo of the 4BarBot and a screenshot from a simulation	15
3.2	General views of the 4BarBot's leg	16
3.3	General views of the 4BarBot's leg	16
3.4	Electronic components of the 4BarBot	17
4.1	Rotation in place diagram	20
4.2	Trajectory generation in 4 different motions	22
4.3	Kinematics schema from the robot base to an end effector of a leg	23
4.4	Four-bar linkage of a leg XZ plane	25
4.5	Four-bar linkage of a leg YZ plane	27
5.1	System architecture of the quadruped controller	31
5.2	An architecture of <i>ros2_control</i> [ROS25]	33
5.3	Photos of a visualization in RViz2 and plots in <i>plot_juggler</i>	35
6.1	Statistics of joint position error while going forward	37
6.2	Statistics of foot position error during walking forward	38
6.3	Statistics of foot position error during walking sideways without ground	39
6.4	Statistics of foot position error while rotating clockwise	39
6.5	Rotating counterclockwise - grounded	40
A.1	Trajectory following of a q_0 joint.	42
A.2	Trajectory following of a q_1 joint.	43
A.3	Trajectory following of a q_2 joint.	43
A.4	Trajectory following of a position of foot in the task space while going forward (among X axis).	44
A.5	Trajectory following of a position of foot in the task space while going sideways (among Y axis).	45
A.6	Trajectory following of a position of foot in the task space while rotating clockwise.	46

A.7 Trajectory following of a position of foot in the task space while rotating counterclockwise.	47
A.8 Trajectory following of a q_0 joint.	48
A.9 Trajectory following of a q_1 joint.	48
A.10 Trajectory following of a q_2 joint.	49
A.11 Trajectory following of a position of foot in the task space while going on the ground forward (among X axis).	50
A.12 Trajectory following of a position of foot in the task space while going on the ground sideways (among Y axis).	51
A.13 Trajectory following of a position of foot in the task space while rotating on the ground clockwise.	52
A.14 Trajectory following of a position of foot in the task space while rotating on the ground counterclockwise.	53

List of Tables

3.1	Key specifications of the RMD-X8-Pro-H 1:6 actuator	17
4.1	Denavit-Hartenberg Table	24
6.1	Statistics of joint position error while going forward	37
6.2	Statistics of Position Errors during walking forward	38
6.3	Statistics of Position Errors during walking sideways	38
6.4	Statistics of Position Errors while rotating clockwise	39
6.5	Statistics of Position Errors while rotating counterclockwise	40

Chapter 1

Introduction

This chapter provides an introduction to the context and background of the thesis. It begins by discussing the motivation behind using legged locomotion, particularly its advantages over wheeled systems in navigating complex and unstructured environments. An overview of existing commercial and research quadruped robots is presented to illustrate the current state of the field. The chapter then introduces the 4BarBot, a custom-built quadruped robot. Finally, it outlines the purpose and scope of the thesis within the context of robotic gait control and evaluation.

1.1 Motivation

Robots that can move freely from one place to another in their environment are called mobile robots. Unlike stationary systems, they are designed to perform tasks in different locations [Tza13]. The way they move depends on their construction. Most mobile robots use wheels or tracks, but legs offer advantages in rough terrain because they can select isolated footholds that optimize support and traction, unlike wheels that require a continuous path. Legs also provide active suspension, allowing the robot's body to move smoothly even on uneven surfaces, and enable stepping over obstacles that wheeled robots cannot overcome [Rai86].

Legged robots move by coordinating the motion of their legs to lift, swing, and place them on the ground. This process is more complex than rolling on wheels because it requires maintaining balance and synchronizing multiple limbs. To control this movement, robots use walking control algorithms. These algorithms decide when and how each leg should move to achieve stable and efficient walking.

Some legged robots utilize parallel or hybrid serial-parallel mechanisms to benefit from the advantages of both architectures, such as high force-to-weight ratio, reduced structural inertia, and expanded workspace [GKY18]. It has also been shown that parallel mechanisms can offer lower energy consumption than their serial counterparts, making them suitable for practical bipedal robots [MYMO00].

Quadruped robots are legged robots that walk using four legs. This design offers a good balance between stability and mobility. With four legs, the robot can support itself on three legs while moving the fourth, which helps maintain balance during walking. Quadruped robots can also move using dynamic gaits, where two legs move simul-

taneously, allowing faster and more efficient locomotion. Because of these abilities, quadruped robots are widely used in research and industry to navigate challenging environments better than wheeled robots or those with fewer legs.

The design process of quadruped robots often begins with the development of locomotion strategies. Gaits themselves can be categorized as periodic or aperiodic; periodic gaits include both static and dynamic variants, while aperiodic gaits are typically used for traversing irregular terrain [XHC⁺22]. Aperiodic gaits, also referred to as free gaits, do not rely on fixed timing or duty factors, allowing for greater adaptability when navigating uneven or unpredictable environments [MF68]. Periodic gaits, on the other hand, follow a repeating time-based pattern. Although they may not capture all aspects of motion, they are effective for modeling regular tasks like walking, running, or hopping, and are widely used in studies of passive and natural locomotion [RBCG17].

Considering the numerous advantages of legged locomotion—such as terrain adaptability, stability, and agility—The Mechatronics and Robotics Laboratory (L7 building B5) which operates within the Mechatronics and Theory of Mechanisms Team [Dep25] decided to design and develop a quadruped robot based on a four-bar linkage mechanism. This architecture offers mechanical simplicity and potential benefits in weight reduction and control robustness. In this thesis, it is investigated how such linkage-based mechanisms perform in practice by analyzing their effectiveness in executing walking behaviors under various locomotion scenarios.

1.2 Commercial and research quadruped robots

Quadruped robots have become increasingly relevant in both commercial and academic contexts due to their versatility in unstructured environments. This section presents an overview of selected state-of-the-art quadruped platforms, highlighting their unique characteristics and areas of application. Figure 1.1 showcases four prominent robots that represent both commercial products and research prototypes.

Spot, developed by Boston Dynamics [Bosnd], shown in Figure 1.1a, is a commercially available quadruped robot designed for industrial inspection, security, and logistics. It features robust perception systems, autonomy, and adaptive locomotion capabilities, making it suitable for real-world deployment.

ANYmal D, presented in Figure 1.1b, is a quadruped robot created by ANYbotics [ANYnd]. It is designed for autonomous inspection in industrial facilities, offering advanced navigation, robust hardware, and high mobility in harsh environments.

Solo, depicted in Figure 1.1c, is a lightweight and fully open-source research robot developed by PAL Robotics [Rob23]. Its minimalistic design and modularity make it ideal for experimental validation of control algorithms and hardware prototyping in academic research.

HyQ, shown in Figure 1.1d, is a hydraulically actuated quadruped robot developed in collaboration between the University of Genoa and the Italian Institute of Technology [Sem10]. It serves as a research platform for dynamic locomotion, offering high torque performance and precise motion control for testing bio-inspired and model-based control strategies.

These platforms show the variety of design approaches and uses in quadruped robots, from commercial models to research prototypes. Different control methods, mechanical designs, and applications highlight the importance of ongoing research in this field. In particular, the comparative study of walking algorithms in this thesis helps to better understand locomotion control and supports the design of control systems for future robots like the 4BarBot, which has a slightly different mechanical structure and is tested for locomotion performance.



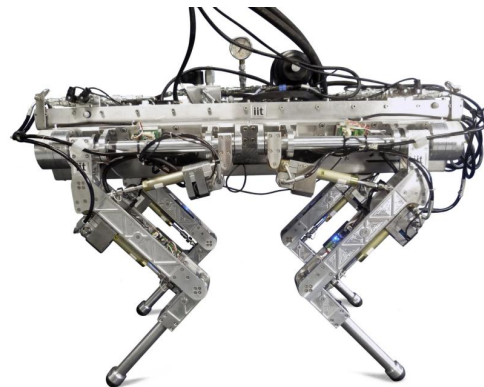
(a) Spot (Boston Dynamics) [[Bosnd](#)]



(b) ANYmal D (ANYbotics) [[ANYnd](#)]



(c) Solo (PAL Robotics) [[Rob23](#)]



(d) HyQ (UoG and IIT) [[Sem10](#)]

Figure 1.1: Four popular quadruped robots: Spot, ANYmal D, Solo, and HyQ.

1.3 Introduction to 4BarBot

4BarBot (Figure 1.2) is a quadruped robot that features four legs, with each leg driven by three separate actuators to enable coordinated movement in the task space. The robot was designed in The Mechatronics and Robotics Laboratory (L7 building B5) which operates within the Mechatronics and Theory of Mechanisms Team [[Dep25](#)]. The mechanical design of the legs incorporates a four-bar linkage mechanism, which contributes to the control of limb positioning and motion. Each actuator is powered by a brushless motor, with dedicated motor controllers, offering a range of control modes including position, velocity, and torque control. To monitor the robot's state during

operation, the system is equipped with sensors such as inertial measurement units, which provide data on orientation and motion dynamics and encoders, which allows to measure velocity and position of a shaft of a motor.

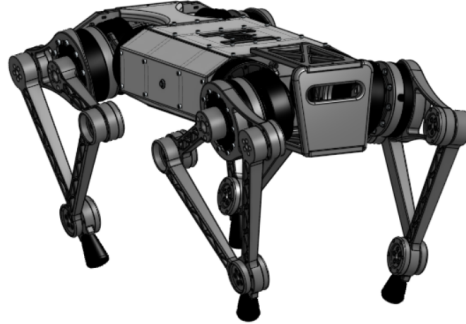


Figure 1.2: CAD model of the 4BarBot

1.4 Objectives of the Thesis

This thesis focuses on evaluating a controller of robot motion in two contrasting scenarios: when the robot is suspended in the air (without ground contact) and when it is walking on the ground with contact forces present. This comparative study aims to evaluate how correctly the control algorithm enables the robot's limbs to follow the desired trajectories. Accurate trajectory tracking is crucial for ensuring coordinated motion, which directly influences the robot's ability to walk correctly under varying environmental conditions.

To support this analysis, a complete control system is implemented in C++ and Python using the ROS 2 framework [MFG⁺22]. The robot's walking behavior is tested in a simulated environment built with MuJoCo [TET12]. The control system of the 4BarBot is managed through the `ros2_control` interface [KT15].

A key part of the study involves tuning the impedance control parameters and analyzing the robot's ability to follow predefined trajectories. Both configuration space and task space are examined to evaluate joint motion, end-effector accuracy, and generated torques.

In addition to the simulation framework, the project incorporates hardware components such as actuators and sensors for motion execution and data collection. Real-time visualization and monitoring tools are also introduced to support debugging, analysis, and validation of the walking algorithms.

Chapter 2

Background and Fundamentals

This chapter provides the theoretical foundation necessary for understanding the control and motion of legged robots such as the 4BarBot. It covers the basics of robot kinematics, methods for solving inverse kinematics, and an overview of various control strategies including impedance control, model predictive control, and reinforcement learning. Additionally, tuning methods for PD controllers are introduced, which are essential for ensuring stable and responsive behavior in robotic systems.

2.1 Gaits

A gait refers to the specific sequence and timing with which a quadruped lifts and places its legs during locomotion [FPW⁺24]. In both animals and legged robots, different gaits—such as walk, trot, pace, or gallop—allow adaptation to varying speeds and terrain complexity, balancing efficiency, stability, and agility. Gaits are typically categorized as symmetrical (e.g., walk, trot) or asymmetrical (e.g., gallop, bound), each defined by distinct leg coordination patterns (see Figure 2.1). In quadruped robotics, control algorithms are designed to execute the desired footfall sequence defined by a gait scheduler, ensuring that the limbs move according to the timing and phasing specified by the selected gait.

The trot is a dynamic and periodic gait in which the robot alternates its diagonal leg pairs during motion—meaning that the front left leg moves in sync with the rear right leg, and vice versa. At any moment, one diagonal pair is in the stance phase (in contact with the ground), while the other is in the swing phase (moving through the air). This coordination allows for balanced and efficient movement, particularly at medium speeds. Due to its symmetry, stability, and relevance for legged locomotion control, the trot gait is selected as the basis for all experiments presented in this work.

2.2 Control algorithms

Modern quadruped robots rely on a variety of control algorithms to ensure stable and efficient locomotion. These include traditional model-free strategies such as PID controllers, which offer simplicity and ease of implementation, as well as more

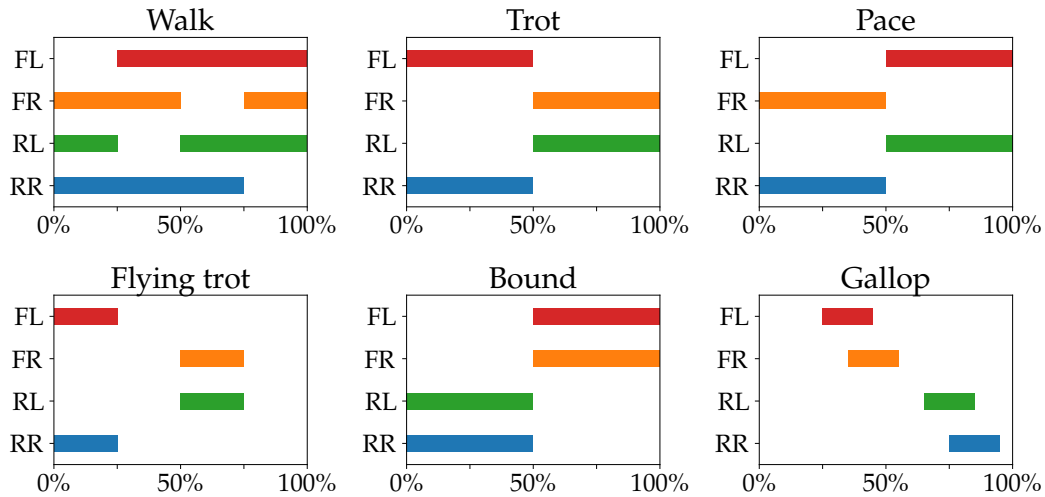


Figure 2.1: Periodic gait patterns of all four legs: FL - front left, FR - front right, RL - rear left, RR - rear right [FPW⁺24]

advanced methods like Model Predictive Control (MPC), which enable dynamic, real-time trajectory planning based on predictive models. Additionally, machine learning techniques, particularly reinforcement learning (RL), allow robots to adapt their gaits autonomously in complex and unstructured environments. A comprehensive review of these approaches highlights their respective strengths, challenges, and applicability depending on the robot's architecture and operating conditions [HRM⁺25].

2.2.1 Impedance control

Impedance control is a method used to regulate the interaction between a robot and its environment by controlling the dynamic relationship between position and force. Unlike traditional position control, which works well in structured environments with little contact, impedance control allows robots to actively adapt their compliance such as stiffness and damping when interacting with uncertain or constrained environments. This is achieved by using feedback to emulate virtual mechanical properties at different points of the robot, enabling it to respond flexibly to external forces. Active impedance control offers the advantage of adjusting these properties in real-time, which is essential for tasks like legged locomotion on varying surfaces [Foc13]. Importantly, the implementation of impedance control is directly based on PID control principles, where proportional, derivative, and sometimes integral components are tuned to achieve the desired dynamic response. In practice, the stiffness and damping terms in impedance control correspond to the proportional and derivative gains in a PID controller, making it a natural extension of classical control strategies.

2.2.2 Model Predictive Control

Model Predictive Control (MPC) is a control strategy that systematically handles constraints during both the design and implementation phases. It is flexible regarding the choice of system model, objective function, and constraints, making it well-suited to reflect various performance criteria. MPC uses a process model to predict future system behavior and optimize control actions accordingly. This versatility and ability to incorporate constraints have made MPC widely successful, especially in complex industrial applications [GPM89]. The Cheetah robot uses Model Predictive Control (MPC) to stabilize a wide range of gaits, including those with complex orientation dynamics. Its controller solves a single convex optimization problem based on a 3D model with 12 degrees of freedom. This solution directly computes motor torques to stabilize the robot without needing an additional feedback controller. The robot achieves a maximum yaw rate of $180 \frac{\text{deg}}{\text{s}}$ and a linear velocity of $3.0 \frac{\text{m}}{\text{s}}$ during a fully 3D gallop, representing one of the fastest gallops for an electrically actuated robot of its scale [DCWK⁺18].

2.2.3 Reinforcement learning

Reinforcement learning is used in this work due to its ability to generate robust locomotion policies that transfer well from simulation to real robots. As shown in the Solo12 experiments, adding random noise during training helps the learned policy handle modeling errors without needing detailed actuator models. This approach enables stable walking on various terrains and supports the development of generalizable controllers for quadruped robots [ALF⁺23].

2.2.4 Conclusions

Among the various control strategies explored, impedance control based on PID regulation is selected for this project. Its simplicity, low computational cost, and ease of implementation make it a practical choice for early-stage testing and algorithm tuning. Despite being less adaptive than advanced methods like MPC or reinforcement learning, impedance PID control can still provide satisfactory locomotion performance, especially when combined with well-designed mechanical structures and proper gain tuning. This makes it an ideal baseline approach for quadruped walking control in simulation and future deployment.

2.3 Impedance Controller tuning

The impedance controller builds upon the classical PID control, where the proportional and derivative gains correspond to virtual stiffness and damping, respectively. This relationship means that tuning an impedance controller involves selecting appropriate K_p and K_d values to regulate how the robot responds to external forces during motion.

Although PID controllers are widely used, their effectiveness depends on proper gain tuning. Without it, the system may become unstable, oscillatory, or sluggish. A common tuning method involves analyzing the step response to evaluate system characteristics such as rise time, overshoot, and settling time. In this work, tuning focuses specifically on the impedance controller to ensure accurate and stable motion during locomotion tasks.

The Cohen–Coon method is a classical PID tuning technique based on the step response of a system. It assumes that the process can be approximated by a first-order plus time delay (FOPTD) model:

$$G(s) = \frac{K}{Ts + 1} e^{-Ls} \quad (2.1)$$

Here, $G(s)$ is the transfer function of the system approximated as a first-order plus time delay (FOPTD) model. The parameter K is the steady-state gain of the system, T is the time constant representing the system's response speed, and L is the time delay that accounts for the system's latency before reacting to input changes. This model forms the basis for the Cohen–Coon PID tuning method [CC22].

2.4 Kinematics

Kinematics describes the motion of robot components without considering the forces behind it. In robotics, it is used to calculate the position and orientation of a robot's limbs or end-effector based on movements of joints. A widely used method for modeling robotic mechanisms is the Denavit-Hartenberg (DH) convention. It assigns coordinate frames to each link and uses four parameters to define the spatial relationship between joints. This approach simplifies the computation of forward kinematics [TM21].

These parameters define the geometric relationship between two adjacent links and are used to build transformation matrices between their coordinate frames. The meaning of each parameter is as follows:

- θ_i : Joint angle (rotation about the z_{i-1} -axis).
- d_i : Link offset (translation along the z_{i-1} -axis).
- a_i : Link length (translation along the x_i -axis).
- α_i : Link twist (rotation about the x_i -axis).

Using these parameters, it is possible to construct the following homogeneous transformation matrix that relates the coordinate frames of two consecutive links:

$$A_{i-1}^i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (2.2)$$

This formulation standardizes the representation of transformations and is widely used in robotic kinematics. In this case, due to the presence of a four-bar linkage (closed kinematic chain) at the end of the leg, the standard DH algorithm cannot be directly applied using measurements from the sensors. Instead of using the position of the third joint q_2 , the position of the passive joint from the four-bar linkage q_{p3} is used.

2.5 Inverse kinematics

Inverse kinematics (IK) algorithms are essential for controlling redundant robotic systems such as humanoid or quadruped robots. These methods compute joint configurations that achieve a desired end-effector position and orientation. One widely used class of algorithms is resolved motion rate control, which converts task-space velocity commands into joint-space velocities.

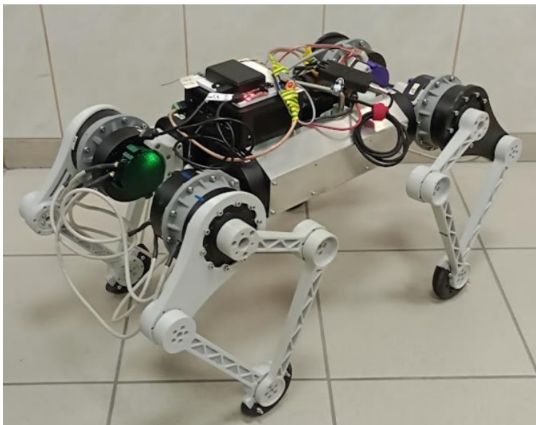
A classical approach in this category is the pseudo-inverse method developed by Liegeois [lie77]. This method is computationally efficient and suitable for real-time control, but it may result in undesirable motion in the null space of the Jacobian. Such extraneous motion can lead to inefficiencies or instability, especially in highly redundant systems.

The geometric approach to inverse kinematics is based on elementary trigonometric principles. It involves analyzing the robot's structure as a series of interconnected triangles and angles, allowing closed-form solutions for joint variables. This method is particularly effective and intuitive for manipulators with a small number of DOF, where the spatial relationships between joints and links can be directly expressed through geometric constraints. The geometrical approach is chosen in the case of the 4BarBot because the pseudo-inverse method is commonly used in serial robots rather than parallel ones.

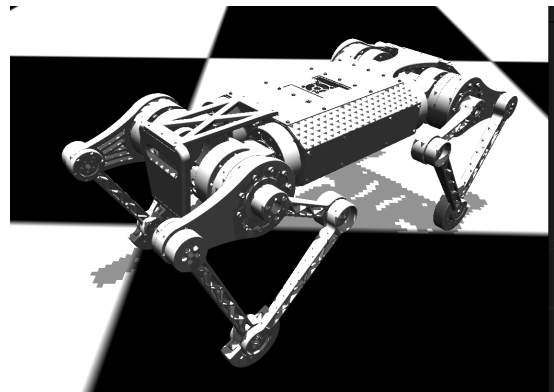
Chapter 3

Construction of 4BarBot robot

This chapter provides a comprehensive overview of the 4BarBot robot's hardware architecture. It begins by detailing the mechanical construction, including both the main body and the unique design of the robot's legs. The chapter then presents the sensing and actuation systems, focusing on the brushless DC motors used in the legs, their corresponding motor controllers, and the inertial measurement unit (IMU). The photo of the quadruped robot and a screenshot of a simulation is shown in Figure 3.1.



(a) Real 4BarBot



(b) Simulated 4BarBot

Figure 3.1: Photo of the 4BarBot and a screenshot from a simulation

3.1 Body construction

The main body of the 4BarBot robot is constructed using CNC-cut aluminum sheets, providing a lightweight yet rigid frame. Some elements of the robot, such as supports, are manufactured using 3D printing technology. Inside the body, the central electronics are mounted. This includes a lithium-polymer (Li-Po) battery that powers the entire system, a DC-DC converter ensures regulated voltage for all components, and an on-board computer responsible for high-level control algorithms, data acquisition, and communication with sensors and actuators. The diagrams of a whole robot construction are shown in Figure 3.2.

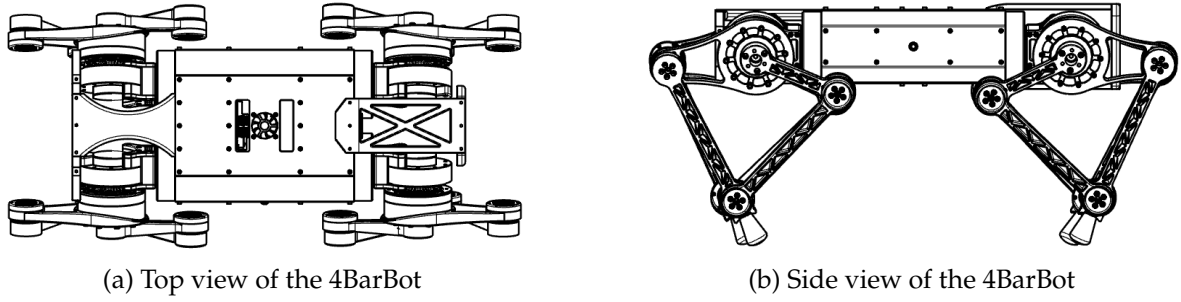


Figure 3.2: General views of the 4BarBot's leg

3.2 Leg construction

Each link of the 4BarBot's leg is 3D-printed. Each element is mounted using screws what enables to fast replacement in case any damage. There are 3 motors with 3 motors controllers for every leg. The first motor enables sideways movement of the leg. The second motor controls the rotation of a four-bar linkage mechanism, while the third motor actuates the internal motion of that linkage. Four-bar linkage introduces 3 passive joints which are built using double bearings and freely rotates. The diagrams of a leg construction are shown in Figure 3.3.

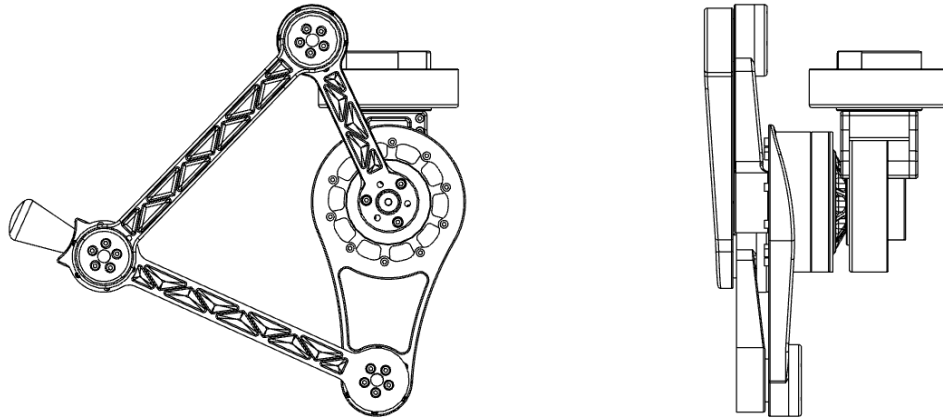


Figure 3.3: General views of the 4BarBot's leg

3.3 Brushless DC Motors in Legged Robotics

Brushless DC (BLDC) motors are synchronous machines in which the magnetic fields of the stator and rotor rotate at the same frequency. Unlike induction motors, they do not exhibit slip, allowing for precise control and efficient torque generation even under variable loads [Yed03]. Their high power density, smooth operation, and compact size make them ideal for legged robots.

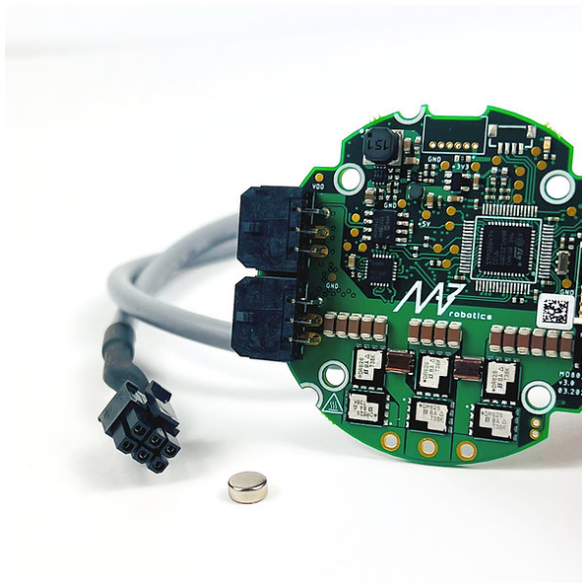
The actuators are based on the **RMD-X8-Pro-H 1:6** module, which integrates a

BLDC motor with a hollow helical gearbox. It provides 8 Nm of nominal torque and reaches 220 rpm at no load, offering a balance of strength and speed in a compact, 780 g design. This makes it well-suited for dynamic and repetitive motion in quadruped locomotion [MyA25].

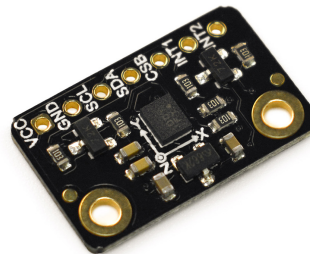
Parameter	Value
Motor type	Brushless DC (BLDC)
Gear ratio	1:6
Gear type	Hollow helical reducer
Nominal torque	8 Nm
No-load speed	220 rpm
Rated output power	135 W
Weight	780 g

Table 3.1: Key specifications of the RMD-X8-Pro-H 1:6 actuator

3.4 Motor controllers



(a) MD80 v3.0 motor controller [MAB25]



(b) 9DOF BMX160 IMU [DFRnd]

Figure 3.4: Electronic components of the 4BarBot

The **MD80 V3.0** by MAB Robotics, shown in Figure 3.4a, is a high-performance brushless motor controller designed for mobile robotics applications. It supports up to 48 V DC and 80 A peak motor current, making it suitable for demanding platforms such as quadruped robots, exoskeletons, and drones. In this project, it is used to control each BLDC motor, providing position, velocity, impedance, and torque control modes. The controller integrates a 14-bit absolute encoder and supports CAN-FD, CANopen, and USB communication, along with inputs for temperature sensors and configurable GPIOs [MAB25].

3.5 IMU

The sensor 9DOF BMX160, shown in Figure 3.4b, features two sensors: the BMX160 and BMP388. The BMX160 is a 9-axis sensor capable of measuring acceleration (± 2 g to ± 16 g) and angular position (gyroscope) with ranges from $125^\circ/\text{s}$ to $2000^\circ/\text{s}$. The BMP388 handles temperature measurements from 0°C to 65°C and atmospheric pressure from 300 hPa to 1250 hPa [DFRnd].

Chapter 4

Theory of quadruped trot gait

This chapter provides the theoretical background necessary to implement walking algorithms in a quadruped robot. It begins by explaining how the robot's motion is organized in time through planned sequences of leg movements known as gaits. The principles of leg movement are then described from a geometric perspective, focusing on how the legs are positioned and how their motion can be calculated to follow a desired path. To achieve precise control of each leg, the mathematical formulas used to determine the required joint angles is discussed. The chapter concludes with an overview of a control method that enables the robot to respond compliantly to its environment by adjusting stiffness and damping of joints, which is essential for stable and adaptive locomotion.

4.1 Gait scheduler

The base link frame, denoted as $\mathcal{F}_b$, is a fixed reference frame attached to the robot's main body. Each of the four legs operates in its own local coordinate frame, denoted as $\mathcal{F}_{li}$ for the i -th leg, where $i = 1, 2, 3, 4$. The trajectory generation calculates the foot positions of a quadruped robot in the base link frame $\mathcal{F}_b$ during trotting.

The step motion is divided into two phases: swing and stance. During the swing phase, the foot follows a predefined sinusoidal trajectory in a selected direction. In the stance phase, the foot remains in contact with the ground and moves backward relative to the body, effectively pushing against the terrain. As a result, the robot's base changes its position with respect to the global frame $\mathcal{F}_g$.

The motion is parameterized by the number of ticks (time steps), initial positions (x_0, y_0) for each leg, and motion deltas $(\Delta x, \Delta y, \Delta z, \Delta \omega)$. Here, $\Delta x, \Delta y$ represent linear displacements, $\Delta \omega$ - angular displacement, while Δz_{stance} and Δz_{swing} represents amplitude of sinusoidal function. Each leg has a phase offset to simulate the alternating gait of a quadruped.

To enable in-place rotation of the robot, each foot trajectory must be updated relative to the robot's center, based on the desired angular displacement $\Delta \omega$. This motion assumes the robot performs a small yaw rotation around its central vertical axis without translating its body. Geometrically, this process is illustrated in Figure 4.1, where each

leg endpoint is rotated around the origin of the robot's base frame $\mathcal{F}_b$ to compute the new target positions.

Let $L_{i0} = (x_0, y_0)^T$ where $i = 1, 2, 3, 4$ for front left (FL), front right (FR), rear left (RL), and rear right (RR) denote the initial position of the i -th foot with respect to the robot's body center $\mathcal{F}_b$. The distance r from the center and the angle ϕ between the leg's foot and the X -axis are computed as

$$r = \sqrt{x_0^2 + y_0^2}$$

$$\phi = \arctan \frac{y_0}{x_0}.$$

After applying the desired in-place rotation $\Delta\omega$, the new position of the foot becomes

$$x' = r \cos(\phi + \Delta\omega)$$

$$y' = r \sin(\phi + \Delta\omega).$$

The change in foot position is then

$$\Delta\omega_x = x' - x_0$$

$$\Delta\omega_y = y' - y_0.$$

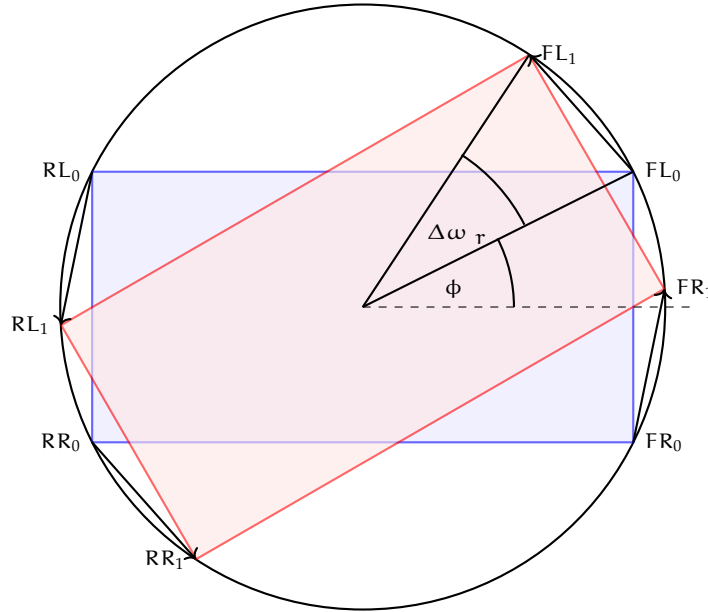


Figure 4.1: Rotation in place diagram

This method ensures that each foot moves along a circular arc around the robot's center, maintaining the same radial distance but shifted by the angular offset $\Delta\omega$, effectively rotating the robot's stance. For translational movements, the foot trajectory is updated by linearly shifting each foot's reference position in the x and y directions. This is achieved by adding the desired displacement values Δx and Δy to the initial coordinates of each foot.

$$\begin{aligned}
x(t) &= \begin{cases} x_0 + (\Delta x + \Delta \omega_x) \cdot \frac{t}{T}, & 0 \leq t < T \\ x_0 + (\Delta x + \Delta \omega_x) - (\Delta x + \Delta \omega_x) \cdot \frac{t-T}{T}, & T \leq t < 2T \end{cases} \\
y(t) &= \begin{cases} y_0 + (\Delta y + \Delta \omega_y) \cdot \frac{t}{T}, & 0 \leq t < T \\ y_0 + (\Delta y + \Delta \omega_y) - (\Delta y + \Delta \omega_y) \cdot \frac{t-T}{T}, & T \leq t < 2T \end{cases}
\end{aligned}$$

Lifting a leg is implemented by using a sinusoidal function in a swing and stance phase

$$z(t) = \begin{cases} z_0 + \Delta z_{\text{swing}} \cdot \sin\left(\frac{\pi t}{T}\right), & 0 \leq t < T \\ z_0 - \Delta z_{\text{stance}} \cdot \sin\left(\frac{\pi(t-T)}{T}\right), & T \leq t < 2T \end{cases}$$

where z_0 is a leg height, Δz_{stance} is a ground push height and Δz_{swing} is a step height.

Generated trajectories for a foot end-effector are in Figure 4.2 where each of them is created with different input for walking forward, sideways, rotate clockwise and counterclockwise. The generated trajectory looks the same when the robot walks forward and backward, the same when it moves sideways to the left and right, and also the same when it rotates clockwise and counterclockwise.

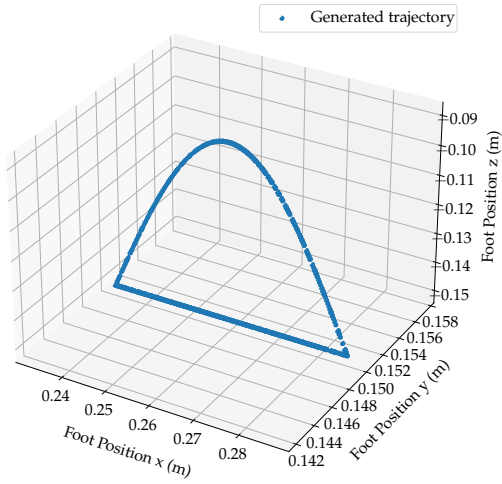
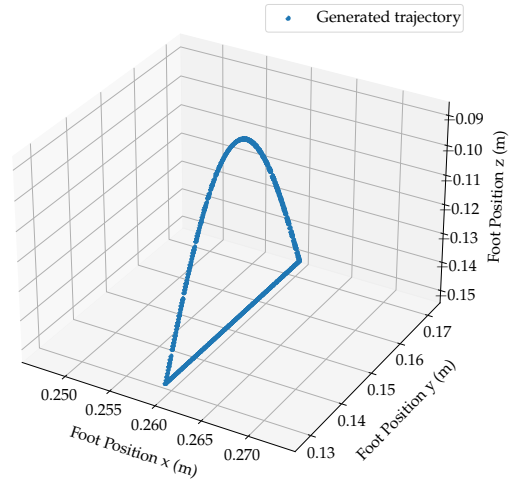
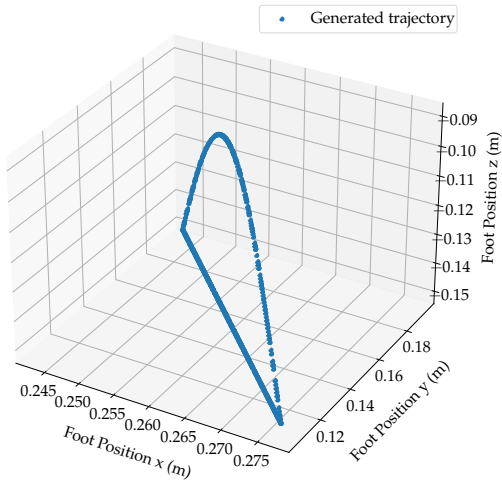
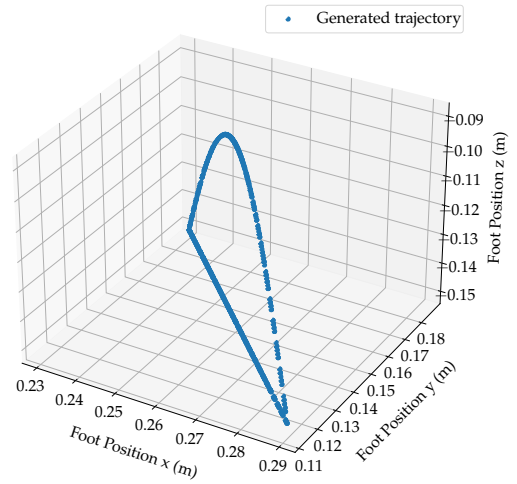
(a) Forward walking (Δx)(b) Sideways walking (Δy)(c) Clockwise rotate (Δx and Δy)(d) Counter clockwise rotate (Δx and Δy)

Figure 4.2: Trajectory generation in 4 different motions

4.2 Leg kinematics

The diagram in Figure 4.3 illustrates the kinematic structure of one leg of the 4BarBot robot. The leg consists of a serial-parallel hybrid architecture. The first two joints, q_0 and q_1 , form a serial chain providing base rotation and linkage actuation. The third joint, q_2 , actuates the four-bar linkage mechanism, which includes an internal joint denoted as q_{p3} . Structural parameters such as a_{10} , a_{11} , d_2 , d_3 , and link lengths l_1 through l_4 define the geometry and relative positioning of each segment.

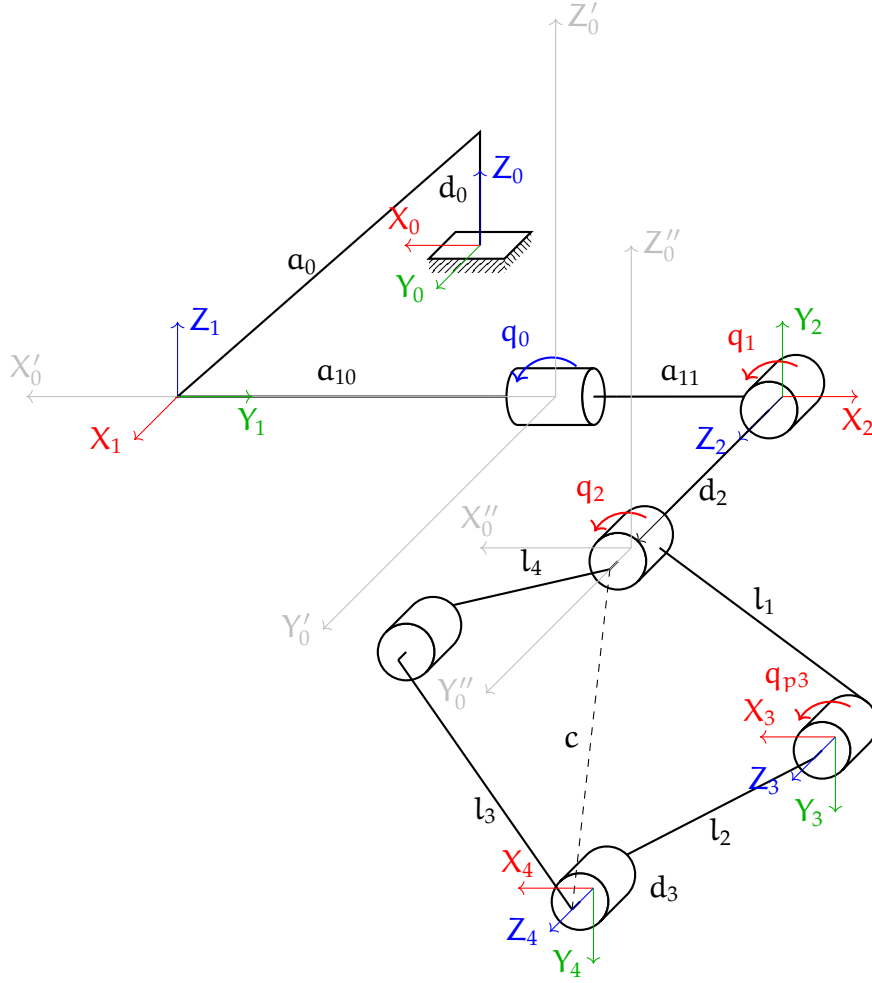


Figure 4.3: Kinematics schema from the robot base to an end effector of a leg

The joint position vector of leg i $\mathbf{q}_i$ in configuration space is defined as:

$$\mathbf{q}_i = \begin{bmatrix} q_{i0} \\ q_{i1} \\ q_{i2} \end{bmatrix}$$

where q_{i0} , q_{i1} , and q_{i2} represent the angular positions of the leg's three joints. The vector of passive joints associated with the four-bar linkage is defined as:

$$\mathbf{q}_p = \begin{bmatrix} q_{ip3} \\ q_{ip4} \end{bmatrix}$$

where q_{ip4} and q_{ip4} represent the internal angles of the passive joints that arise due to the closed-loop kinematic chain of the four-bar mechanism. These values are not controlled directly but depend on the geometry and motion of the active joint q_{i2} .

An i -th leg kinematics $K(\mathbf{q}_i, \mathbf{q}_{pi})$ is composed using 4 transformations:

$$K(\mathbf{q}_i, \mathbf{q}_{pi}) = A_0^1 A_1^2(q_0) A_2^3(q_1) A_3^4(q_{p3})$$

where first transform A_0^1 is a static transformation from the base frame $\mathcal{F}_b$ of the robot to the base of a leg. $A_1^2(q_0)$ and $A_2^3(q_1)$ are calculated in respect to q_0 and q_1 . The last transformation $A_3^4(q_{p3})$ is calculated in respect to the position of passive joint q_{p3} .

In a quadruped robot, legs are positioned symmetrically as front or rear, and left or right. To represent the kinematics using Denavit-Hartenberg parameters, the offsets along coordinate axes need to be adjusted depending on leg position. Multiplying base parameters by ± 1 factors, which depend on the leg's position, simplifies the model and ensures symmetry.

Multipliers for each leg i :

$$m_x(i) = \begin{cases} +1, & \text{if leg } i \text{ is front} \\ -1, & \text{if leg } i \text{ is rear} \end{cases}$$

$$m_y(i) = \begin{cases} +1, & \text{if leg } i \text{ is left} \\ -1, & \text{if leg } i \text{ is right} \end{cases}$$

Modified Denavit-Hartenberg parameters:

$$a'_i = m_x(i) a_i$$

$$d'_i = m_y(i) d_i$$

After modifying the DH parameters according to the leg positions, it becomes possible to systematically construct the Denavit-Hartenberg table (see Table 4.1), which accurately represents the kinematic configuration for each leg of the robot.

Link	θ	d	a	α
A_0^1	$\frac{\pi}{2}$	d'_0	a'_0	0.0
A_1^2	$q_0 - \frac{\pi}{2}$	0.0	$(a'_{10} + a'_{11})$	$\frac{\pi}{2}$
A_2^3	$q_1 + \pi$	d'_2	$l1$	0.0
A_3^4	$q_{p3}(q_2)$	d'_3	$l4$	0.0

Table 4.1: Denavit-Hartenberg Table

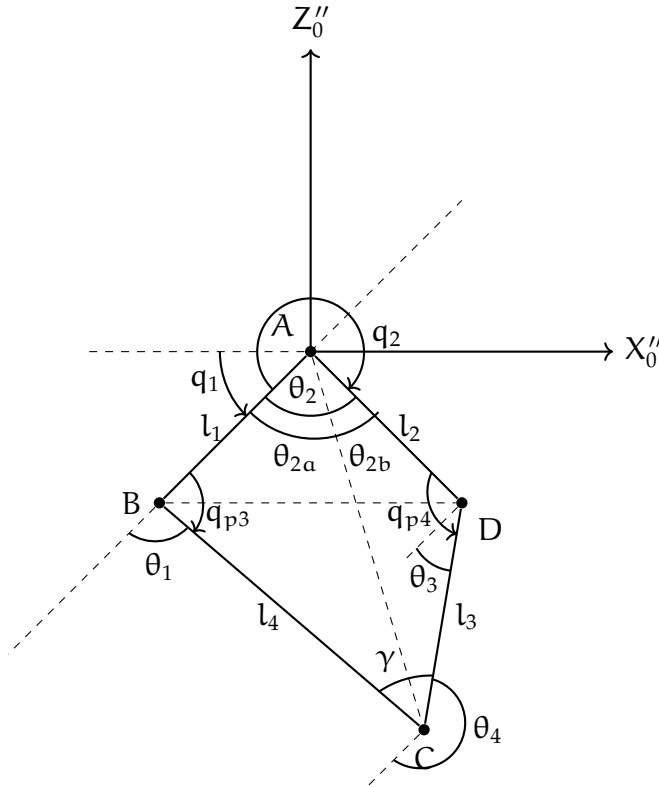


Figure 4.4: Four-bar linkage of a leg XZ plane

4.2.1 Computing passive angle q_{p3}

The constraint function defined in (4.1) arises from the vector loop closure condition of the four-bar linkage. It represents the fact that the sum of vectors forming the closed kinematic chain must return to the starting point. Each term in the equation corresponds to a segment of the linkage, and their vectorial sum in the Cartesian plane must equal zero

$$\Psi = \begin{bmatrix} l_4 \cos \theta_1 + l_1 - l_2 \cos \theta_2 - l_3 \cos \theta_3 \\ l_4 \sin \theta_1 - l_2 \sin \theta_2 - l_3 \sin \theta_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \quad (4.1)$$

This equation ensures that the geometric constraint imposed by the closed-chain structure of the mechanism is satisfied at all times.

The passive joint angles in the four-bar linkage are computed based on the geometry of the mechanism provided in the Figure 4.4. The derivation is based on solving a vector loop that forms a closed kinematic chain.

First, the internal angle θ_2 is related to the actuated joint angle q_2 by

$$\theta_2 = \pi - q_2.$$

Next, the distance between the fixed points B and D is computed using the law of cosines

$$BD = \sqrt{l_1^2 + l_2^2 - 2l_1l_2 \cos \theta_2}.$$

Then, the angle γ between the links l_3 and l_4 is obtained using another application of the law of cosines

$$\gamma = \arccos \left(\frac{l_3^2 + l_4^2 - BD^2}{2l_3l_4} \right).$$

The angle θ_1 is calculated using the tangent-half-angle formula derived from the vector loop equation [Mys12]

$$\theta_1 = 2 \arctan \left(\frac{l_2 \sin \theta_2 - l_3 \sin \gamma}{l_2 \cos \theta_2 + l_4 - l_1 - l_3 \cos \gamma} \right).$$

Similarly, the passive angle θ_3 is given by

$$\theta_3 = 2 \arctan \left(\frac{l_4 \sin \gamma - l_2 \sin \theta_2}{l_1 + l_3 - l_2 \cos \theta_2 - l_4 \cos \gamma} \right).$$

Finally, the passive joints positions q_{p3} and q_{p4} , used for describing the configuration of the mechanism, is expressed as:

$$q_{p3} = \pi - \theta_1 \quad (4.2)$$

$$q_{p4} = \pi - \theta_2 + \theta_3 \quad (4.3)$$

4.3 Leg inverse kinematics

The inverse kinematics of the leg is solved to determine the joint angles required to place the foot at a desired position in the task space provided by a gait scheduler. Inverse kinematics for legs are calculated in the i -th leg base frame $\mathcal{F}_{li}$. The desired position of each leg $\mathbf{x}_{bi}$ is expressed in the base frame $\mathcal{F}_b$.

$$\mathbf{x}_{bi} = \begin{bmatrix} x_{bi} \\ y_{bi} \\ z_{bi} \end{bmatrix} \quad (4.4)$$

To translate it to the leg base frame $\mathcal{F}_{li}$ it is necessary to move the desired point by translation $\mathbf{T}_b^i$:

$$\mathbf{x}_{li} = \mathbf{x}_{bi} - \mathbf{T}_b^i = \begin{bmatrix} x_{li} \\ y_{li} \\ z_{li} \end{bmatrix} \quad (4.5)$$

where

$$\mathbf{T}_b^i = \begin{bmatrix} -(a'_{10} + a'_{11}) \\ a'_0 \\ d'_0 \end{bmatrix}$$

The first configuration variable q_0 corresponds to the lateral abduction/adduction motion of the leg and is obtained by projecting the foot position onto the YZ plane of the leg, as shown in Figure 4.4. This joint determines the sideways orientation of the leg and defines the base rotation for the rest of the inverse kinematics calculation.

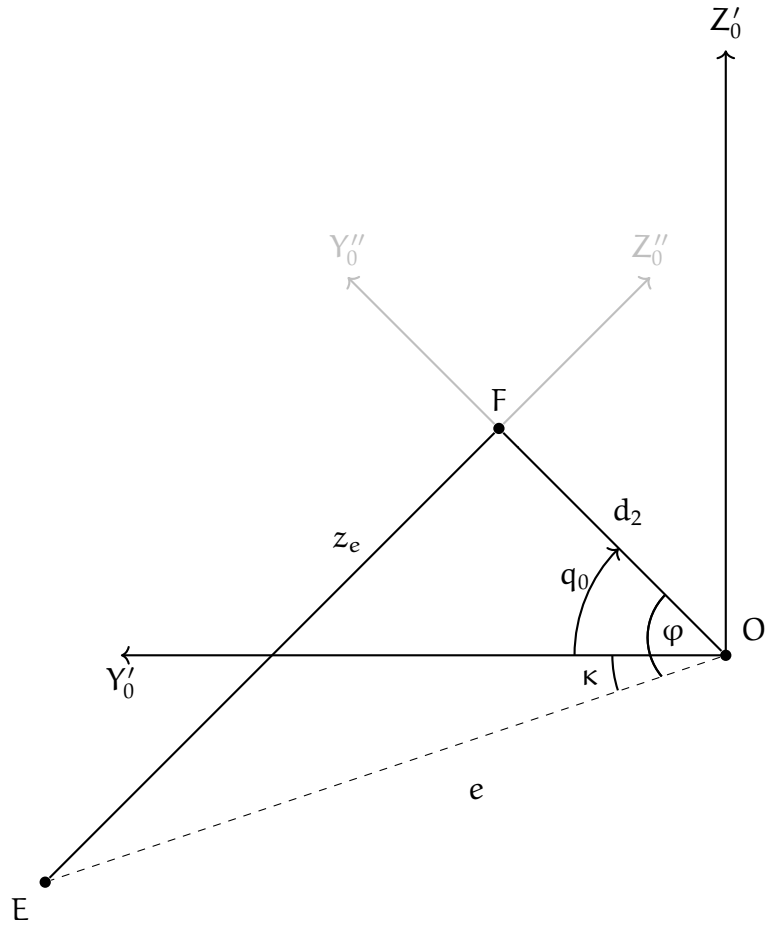


Figure 4.5: Four-bar linkage of a leg YZ plane

The position of point C, which is the projection of the foot onto the YZ plane of the leg in the leg base frame $\mathcal{F}_i$, is defined as

$$\mathbf{E} = \begin{bmatrix} y_{li} \\ z_{li} \end{bmatrix}.$$

The value e represents the Euclidean distance from the hip joint to the projected foot position

$$e = \sqrt{y_{li}^2 + z_{li}^2}.$$

The angle φ is determined using the inverse cosine law, based on the known distance d_2

$$\varphi = \arccos\left(\frac{d_2}{e}\right). \quad (4.6)$$

The angle κ describes the orientation of point C with respect to the Y axis and is obtained from the inverse of tangent function of its coordinates

$$\kappa = \arctan\left(\frac{z_{li}}{y_{li}}\right). \quad (4.7)$$

The first joint angle q_0 is calculated as a sum of the angles φ and κ

$$q_0 = \varphi + \kappa. \quad (4.8)$$

The remaining configuration variables q_1 and q_2 are computed based on the desired position of the foot in the XZ plane of a coordinate frame attached to the base of the four-bar linkage $\mathcal{F}_{i4bar}$. In this setup, the joint variable q_1 is responsible for rotating the entire four-bar mechanism with respect to its frame $\mathcal{F}_{i4bar}$, while q_2 actuates the mechanism by driving one of its input links. This rotation modifies the internal configuration of the four-bar linkage and determines the resulting position of the passive joints. The inverse kinematics of the leg is calculated graphically based on the Figure 4.4.

Firstly the end effector position is changed thanks to the adduction provided by q_0 shown in Figure 4.5. As the result z_e coordinate of C point is

$$z_e = \sqrt{e^2 - d_1^2}.$$

The points C and B represent positions in the xz-plane

$$C = \begin{bmatrix} x_{li} \\ z_e \end{bmatrix}, \quad B = \begin{bmatrix} x_B \\ z_B \end{bmatrix}.$$

Using the law of cosines or Euclidean distances, the geometric constraints are defined as

$$\begin{cases} l_1^2 = x_B^2 + z_B^2 \\ l_4^2 = (x_{li} - x_B)^2 + (z_e - z_B)^2 \end{cases} \quad (4.9)$$

The scalar term Q is calculated as

$$Q = \frac{l_1^2 - l_4^2 + x_{li}^2 + z_e^2}{2z_e}. \quad (4.10)$$

Auxiliary variables are introduced

$$W = Q^2 - l_{AB}^2, \quad T = -\frac{2Qx_{li}}{z_e}, \quad V = 1 + \frac{x_{li}^2}{z_e^2}.$$

The discriminant of the resulting quadratic equation is

$$\Delta = T^2 - 4 \cdot V \cdot W.$$

The possible vertical positions of point B are

$$z_{B1} = \frac{-T + \sqrt{\Delta}}{2V}, \quad z_{B2} = \frac{-T - \sqrt{\Delta}}{2V}. \quad (4.11)$$

Corresponding horizontal positions are

$$x_{B1} = \sqrt{l_1^2 - z_{B1}^2}, \quad x_{B2} = \sqrt{l_1^2 - z_{B2}^2}.$$

The hip joint angle q_1 is computed from the coordinates of point B as

$$q_1 = \pi + \arctan\left(\frac{z_B}{x_B}\right).$$

The angle between links at the distal end is calculated as

$$q_{p3} = \arctan\left(\frac{z_e - z_B}{x_{li} - x_B}\right).$$

The distance c between the origin A and point B is

$$c^2 = x_B^2 + z_B^2.$$

Using the law of cosines, two intermediate angles are derived

$$\theta_{2a} = \arccos\left(\frac{c^2 + l_1^2 - l_4^2}{2cl_1}\right),$$

$$\theta_{2b} = \arccos\left(\frac{c^2 + l_2^2 - l_3^2}{2cl_2}\right).$$

Their sum defines the internal configuration angle

$$\theta_2 = \theta_{2a} + \theta_{2b}.$$

Finally, the second joint angle is calculated as

$$q_2 = \pi - \theta_2 = \pi - \theta_{2a} - \theta_{2b}.$$

Finally, the inverse kinematics is given by

$$\mathbf{q}_i = \text{IK}(\mathbf{x}_{li}) = \text{IK}_0(\mathbf{E}_b - \mathbf{T}_b^i) = \begin{bmatrix} q_{i0} \\ q_{i1} \\ q_{i2} \end{bmatrix} = \begin{bmatrix} \arccos\left(\frac{d_2}{e}\right) - \arctan\left(\frac{z_{li}}{y_{li}}\right) \\ \pi + \arctan\left(\frac{z_B}{x_B}\right) \\ \pi - \theta_{2a} - \theta_{2b} \end{bmatrix}. \quad (4.12)$$

4.4 Impedance controller

The Proportional-Derivative (PD) component of the impedance controller computes the output torque $\tau_i(t)$ for the i -th leg as:

$$\tau_i(t) = K_p \mathbf{e}_i(t) - K_d \mathbf{v}_i(t) + \tau_{iff}. \quad (4.13)$$

Here, $\mathbf{e}_i(t) = \mathbf{q}_d(t) - \mathbf{q}(t)$ is the error vector between the desired position $\mathbf{q}_d(t)$ and the measured position vector $\mathbf{q}(t)$. The vector $\mathbf{v}_i(t)$ represents a velocity of joints and τ_{iff} is a feed-forward torque. K_p and K_d are a diagonal matrices:

$$K_p = \begin{bmatrix} K_{p1} & 0 & 0 \\ 0 & K_{p2} & 0 \\ 0 & 0 & K_{p3} \end{bmatrix} \quad K_d = \begin{bmatrix} K_{d1} & 0 & 0 \\ 0 & K_{d2} & 0 \\ 0 & 0 & K_{d3} \end{bmatrix}$$

where rows are associated with the specific joint.

Chapter 5

Implementation

This chapter presents the software architecture and implementation details of the quadruped robot control system (Figure 5.1). The system is developed using the Robot Operating System 2 (ROS 2), which provides modular tools and communication infrastructure for robotic applications. The chapter begins with an overview of ROS 2 and its role in the project.

Next, the design and functionality of the gait scheduler are described, focusing on its role in generating foot trajectories from velocity commands. This is followed by an explanation of the ROS 2 Control framework, which manages the execution of control loops and hardware communication.

The Quadruped Controller section describes the main control logic that transforms foot trajectories into joint commands, applies inverse kinematics, and integrates impedance control.

Subsequently, the Hardware Interfaces used to communicate with the robot's actuators and sensors are outlined. Finally, the visualization section discusses the use of RViz2 and PlotJuggler for real-time monitoring, debugging, and analysis of robot behavior.

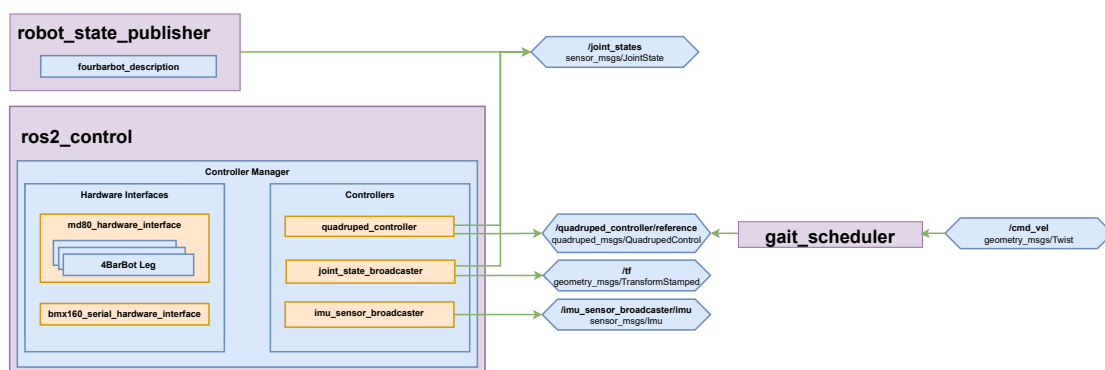


Figure 5.1: System architecture of the quadruped controller

5.1 Robot Operating System 2

ROS 2 (Robot Operating System 2) is an open-source framework that provides a collection of software libraries and tools for developing robotic applications. It builds upon the foundations of ROS 1, offering improved performance, modularity, and support for real-time systems. ROS 2 includes components for hardware abstraction, device drivers, communication, simulation, and algorithm integration. Designed to be scalable and platform-independent, it facilitates the development of robust and flexible robotic systems suited for both research and industrial applications [MFG⁺22].

5.2 Implementation of the Gait Scheduler

The gait scheduler module is implemented in Python using `rcipy` library and serves as a key component of the locomotion system. It receives motion commands in the form of *geometry_msgs/msg/Twist* messages, a standard message type in ROS 2 used for representing linear and angular velocity command vectors in mobile robots. These messages contain three components: linear velocity in the x and y directions, and angular velocity around the z -axis.

Based on the input velocities, the gait scheduler computes reference trajectories for each of the robot's four feet within the robot's local coordinate frame. The computation involves determining both the stance and swing phases of the trotting gait.

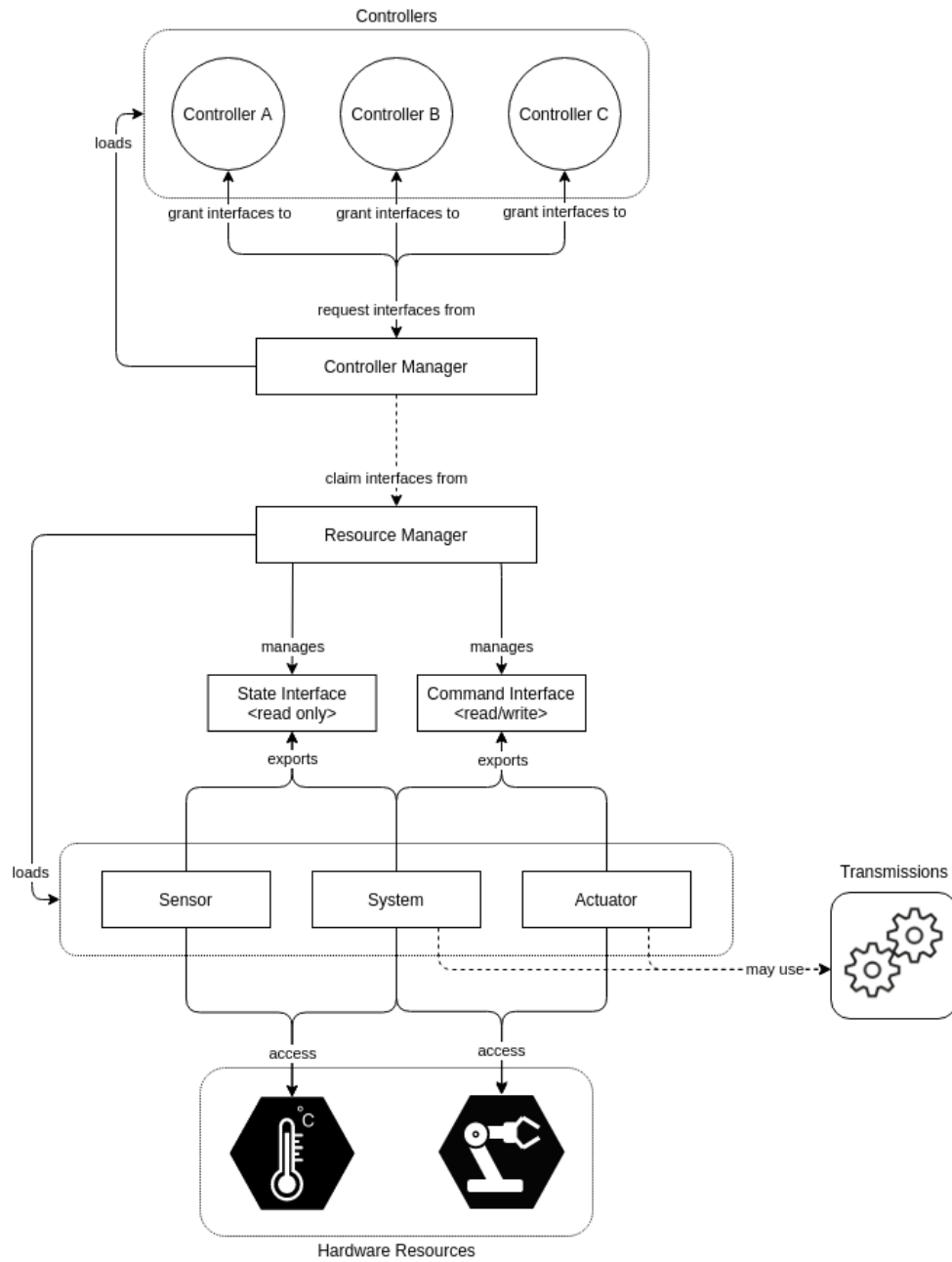
Instead of sending complete trajectories all at once, the gait scheduler publishes discrete trajectory points sequentially. This design choice allows the controller to follow the trajectory point-by-point. It also enables continuous updates and dynamic replanning if new velocity commands are received during locomotion.

The gait scheduler acts as a bridge between high-level velocity commands and low-level foot trajectory tracking, enabling the robot to move in a desired direction while maintaining stability and coordination.

5.3 ROS 2 Control

The `ros2_control` framework is designed for real-time control of robots within the ROS 2 ecosystem [ROS25]. At the core of the framework is the Controller Manager, which serves as the interface between controllers and the hardware abstraction layer. The Resource Manager provides abstraction for physical hardware components and is responsible for managing their state, interfaces, and communication using the `pluginlib` system. This allows hardware such as motors, sensors, and complex robotic subsystems to be reused and integrated flexibly across different robotic platforms.

Controllers in `ros2_control` are implemented based on classical control theory. They calculate system inputs by comparing reference values to measured outputs and act accordingly to reduce the control error. Each controller is implemented as a plugin and follows the ROS 2 lifecycle model, allowing robust state management and real-time performance.

Figure 5.2: An architecture of *ros2_control* [ROS25]

The hardware components within the framework abstract physical devices and expose their functionality through standardized interfaces. This unified architecture enables seamless control-loop execution, where the Controller Manager reads the current state from the hardware, updates all active controllers, and writes the resulting commands back to the hardware in each cycle. These components can be also implemented as communication bridges with simulators.

The 4BarBot is composed using components implemented in *ros2_control* framework such as *bmx160_serial_hardware_interface* which reads measurements from the IMU attached to the computer via USB and shares them with the Control Manager, *md80_hardware_interface* which is used to communicate with MD80 motors controllers via CAN and *quadruped_controller* which receives goal position for feet, computes state of a robot using inverse and forward kinematics, and manages impedance control.

5.4 Quadruped controller

The *quadruped_controller* is responsible for translating high-level foot position references into low-level joint commands suitable for execution by the robot's actuators. Upon receiving target foot positions from the gait scheduler, the controller first verifies their feasibility. This includes checking for the availability of valid inverse kinematics (IK) solutions and ensuring that the requested poses do not lead to kinematic singularities. Once validated, the Cartesian foot positions are transformed into joint space using inverse kinematics, producing a set of joint angles that achieve the desired foot placement. If no valid IK solution is found, the command is ignored and the target position defaults to the last valid pose.

The resulting joint targets are then forwarded to the impedance controller when control torques are generated. Through virtual stiffness and damping mechanisms, the impedance controller enables safe and stable motion execution, particularly important during contact with the ground.

In addition to the primary control functionality, the controller also calculates the positions of passive joints, which are mechanically coupled and not directly actuated. These passive joint values are published to the *joint_states* topic, enabling accurate visualization in RViz 2 and providing insight into the full configuration of the leg mechanism during motion.

Furthermore, the controller publishes diagnostic and visualization data, such as joint angles, end-effector positions, and control signals. This information supports algorithm inspection, debugging, and performance evaluation during experiments.

Finally, the computed joint torque commands are passed to the *md80_hardware_interface* or in case of simulation to the *mujoco_system_plugins*.

5.5 Hardware Interfaces

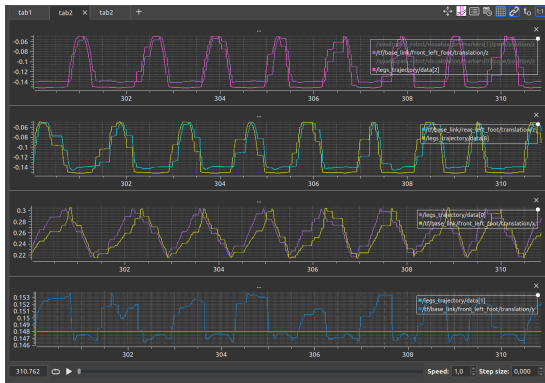
The communication with MD80 motor controllers as well as their configuration is managed in the *md80_hardware_interface*. This plugin inherits a *ros2_control* sys-

tem properties which contains, for state interfaces - position, velocity and torque, for command interface - torque. The hardware interface (HW) uses CANDle C++ library what is the heart of ecosystem on the master controller side [Rob24]. Thanks to the modularity of *ros2_control* this HW can be swapped with the *mujoco_ros2_control*

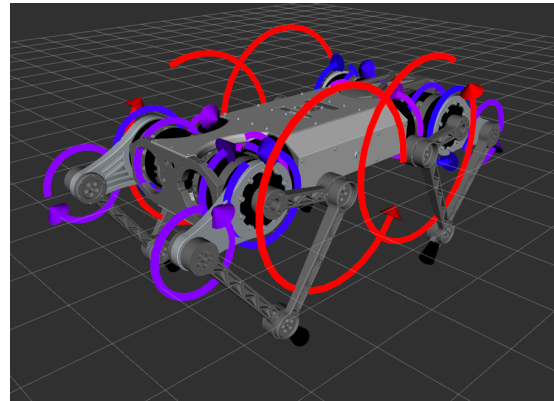
An acquisition of IMU data is performed on a microcontroller through I²C serial communication interface. Measurements such as acceleration, gyration and magnitude are used for estimating robot's orientation. An estimation process is managed by the attitude and heading reference system algorithm (AHRS) [TRR17]. The data is packed and sent to the computer using USB. Packets are received by *bmx160_serial_hardware_interface* and exported as *ros2_control* state interfaces such as a quaternion of rotation, 3D acceleration vector, 3D gyration vector and 3D magnitude vector. The data is published to the ecosystem using *imu_sensor_broadcaster* as *sensor_msgs/msg/IMU* and *sensor_msgs/msg/MagneticField*.

5.6 Visualization

In this work, two tools are used to support the analysis and debugging of the quadruped robot control system. **RViz2** is utilized for real-time visualization of the robot's position and components, enabling monitoring of the kinematic structure's behavior. **PlotJuggler** is employed for plotting data such as positions and forces, which significantly facilitated trajectory analysis and debugging of control algorithms. An example of the visualization setup is shown in Figure 5.3.



(a) Real-time plots in *plot_juggler*



(b) Torques vizualization in RViz2

Figure 5.3: Photos of a visualization in RViz2 and plots in *plot_juggler*

Chapter 6

Impedance Control Analysis

This chapter presents an analysis of impedance control performance in both configuration and task space during various locomotion scenarios. The comparison is made between two key conditions: when the robot is floating (without ground contact) and when it interacts with the ground. By examining these two states across different inputs for gait scheduler, the comparison is aimed to assess how impedance control responds to external disturbances and how accurately the robot tracks desired trajectories.

6.1 Configuration space

Table 6.1 and Figure 6.1 present the tracking error statistics for individual joints while the robot performs a forward locomotion task. The corresponding time-domain plots illustrating joint tracking performance can be found in the Appendix A. The results compare two scenarios: one with the robot in a floating base configuration (no contact with the ground), and the other with ground contact.

As expected, joint q_0 , which remains static during the gait, exhibits the smallest error in both cases. In contrast, joints q_1 and q_2 show higher errors due to their active role in following the varying joint-space trajectory.

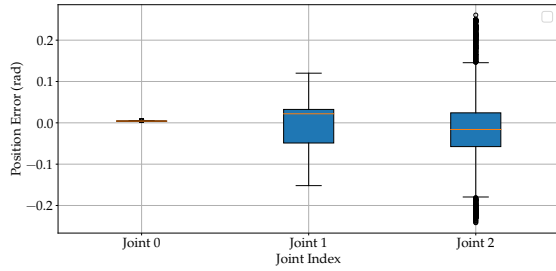
Notably, when ground contact is enabled, the tracking errors increase significantly. They are not only bigger than the floating experiment, but means are shifted over 0 what cannot be treated as white noise for Kalman Filter for example. This behavior is primarily due to external disturbances introduced by interaction with the environment, making trajectory tracking more challenging for the impedance controller.

6.2 Task space

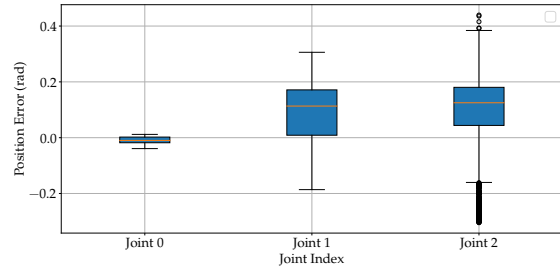
This section presents the statistical analysis of foot position tracking errors during various locomotion modes: forward walking (Table 6.2, Figure 6.2), lateral movement (Table 6.3, Figure 6.3), and in-place rotations—both clockwise (Table 6.4, Figure 6.4) and counter-clockwise (Table 6.5, Figure 6.5). The corresponding time-domain plots are provided in Appendix A.

Joint	Mean (rad)	Median (rad)	Std Dev (rad)	Min (rad)	Max (rad)
Floating					
q_0	0.0043	0.0043	0.0006	0.0035	0.0056
q_1	-0.0023	0.0219	0.0711	-0.1519	0.1201
q_2	-0.0068	-0.0161	0.1208	-0.2416	0.2607
Grounded					
q_0	-0.0095	-0.0098	0.0119	-0.0391	0.0118
q_1	0.0889	0.1133	0.1316	-0.1861	0.3060
q_2	0.0890	0.1251	0.1676	-0.3047	0.4393

Table 6.1: Statistics of joint position error while going forward



(a) Joint position error floating



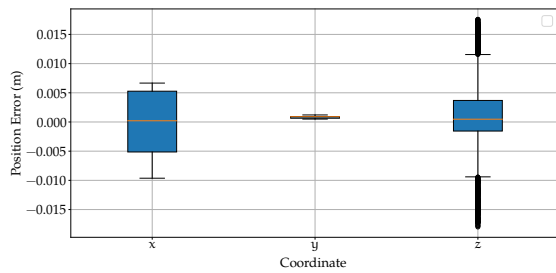
(b) Joint position error grounded

Figure 6.1: Statistics of joint position error while going forward

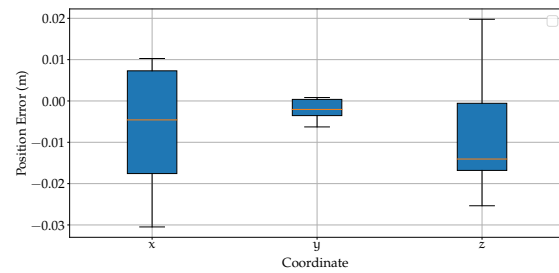
Similar to the behavior observed in the joint space, when the robot is in a floating state (i.e., without ground contact), errors appear only in the coordinates for which the trajectory varies and they are small. In contrast, during ground contact scenarios, the errors are significantly larger. Moreover, on the time-domain plots, one can clearly identify the moments of foot-ground contact, which manifest as abrupt disturbances or deviations in the tracking signal.

Coordinate	Mean (m)	Median (m)	Std Dev (m)	Min (m)	Max (m)
Floating					
x	-0.0004	0.0002	0.0052	-0.0096	0.0067
y	0.0008	0.0009	0.0002	0.0005	0.0012
z	0.0007	0.0005	0.0083	-0.0180	0.0176
Grounded					
x	-0.0059	-0.0046	0.0125	-0.0305	0.0103
y	-0.0018	-0.0020	0.0021	-0.0063	0.0008
z	-0.0084	-0.0141	0.0124	-0.0254	0.0198

Table 6.2: Statistics of Position Errors during walking forward



(a) Walking forward - floating



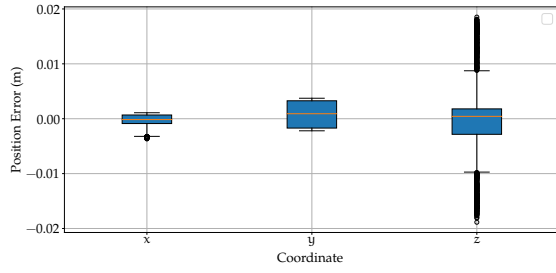
(b) Walking forward - grounded

Figure 6.2: Statistics of foot position error during walking forward

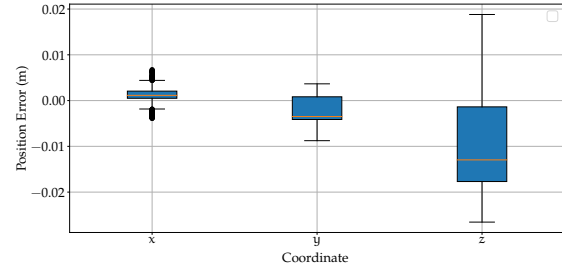
Coordinate	Mean (m)	Median (m)	Std Dev (m)	Min (m)	Max (m)
Floating					
x	-0.0004	-0.0002	0.0012	-0.0037	0.0011
y	0.0008	0.0009	0.0024	-0.0022	0.0037
z	0.0002	0.0004	0.0085	-0.0189	0.0185
Grounded					
x	0.0011	0.0011	0.0019	-0.0039	0.0067
y	-0.0020	-0.0035	0.0028	-0.0088	0.0037
z	-0.0089	-0.0130	0.0120	-0.0266	0.0188

Table 6.3: Statistics of Position Errors during walking sideways

The experiments demonstrates that foot and joint position tracking errors are signifi-



(a) Walking sideways - floating

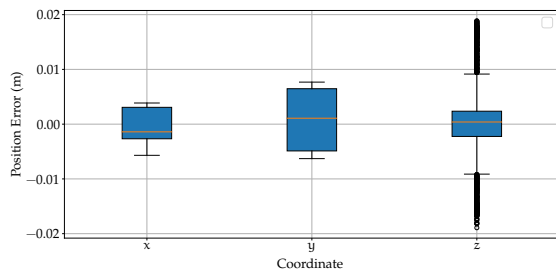


(b) Walking sideways - grounded

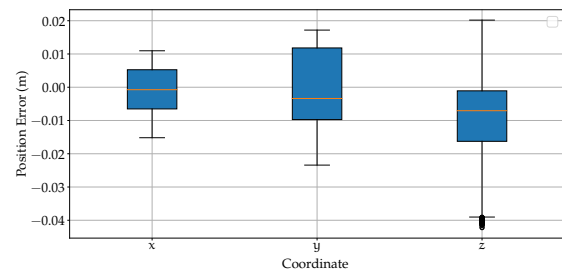
Figure 6.3: Statistics of foot position error during walking sideways without ground

Coordinate	Mean (m)	Median (m)	Std Dev (m)	Min (m)	Max (m)
Floating					
x	-0.0004	-0.0014	0.0029	-0.0057	0.0039
y	0.0008	0.0011	0.0055	-0.0063	0.0077
z	0.0005	0.0004	0.0084	-0.0189	0.0189
Grounded					
x	-0.0010	-0.0007	0.0062	-0.0152	0.0110
y	-0.0004	-0.0034	0.0113	-0.0234	0.0172
z	-0.0078	-0.0071	0.0148	-0.0422	0.0202

Table 6.4: Statistics of Position Errors while rotating clockwise



(a) Rotating clockwise - floating



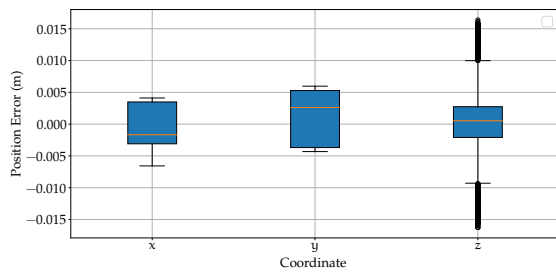
(b) Rotating clockwise - grounded

Figure 6.4: Statistics of foot position error while rotating clockwise

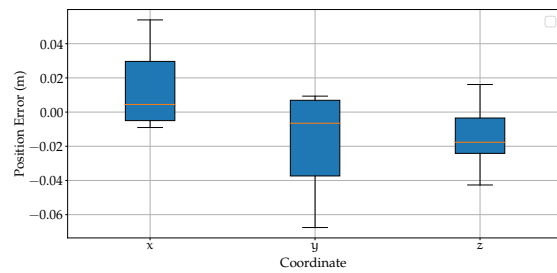
cantly influenced by ground contact conditions. In floating scenarios, errors remain low and are present only along varying trajectory directions, while contact with the ground introduces larger disturbances due to dynamic interactions. These findings confirm the importance of ground-aware control and highlight the challenges of accurate trajectory execution during real-world locomotion.

Coordinate	Mean (m)	Median (m)	Std Dev (m)	Min (m)	Max (m)
Floating					
x	-0.0005	-0.0017	0.0033	-0.0066	0.0041
y	0.0010	0.0026	0.0043	-0.0043	0.0060
z	0.0005	0.0005	0.0078	-0.0163	0.0164
Grounded					
x	0.0125	0.0045	0.0197	-0.0090	0.0539
y	-0.0158	-0.0065	0.0248	-0.0675	0.0093
z	-0.0148	-0.0176	0.0156	-0.0426	0.0161

Table 6.5: Statistics of Position Errors while rotating counterclockwise



(a) Rotating clounterclockwise - floating



(b)

Figure 6.5: Rotating counterclockwise - grounded

Chapter 7

Conclusion

This thesis presented a comparative study of a walking control algorithm for the quadruped robot 4BarBot, focusing on two key scenarios: locomotion without ground contact (floating) and with ground contact (walking). The robot was controlled using a trot gait, a dynamic and symmetric walking pattern commonly used in quadruped locomotion. The goal was to evaluate the effectiveness, stability, and robustness of the impedance-based controller in ensuring accurate trajectory tracking under varying physical conditions.

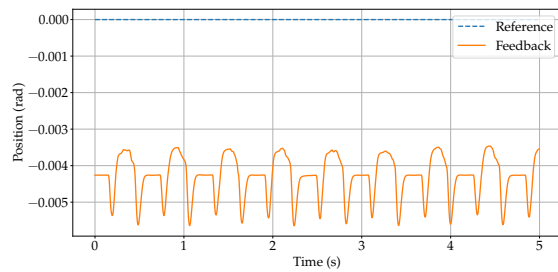
The research began with a comprehensive literature review covering legged locomotion, gait generation, and control strategies, including impedance control. Based on this foundation, appropriate tools and technologies were selected: the ROS 2 framework for modular implementation, Gazebo for simulation, and RViz2 and PlotJuggler for real-time visualization and debugging. A gait scheduler was implemented to generate step trajectories from velocity commands, and a quadruped controller was developed to convert these trajectories into joint commands and send them to the impedance controller via ROS 2 Control. Additionally, custom hardware interfaces were implemented for communication with the md80 motor drivers and the bmx160 IMU sensor, ensuring integration between high-level software control and the robot's physical hardware.

The experiments included various locomotion tasks: forward walking, lateral motion, and rotation, performed in both floating and grounded conditions. The results revealed significantly larger tracking errors when the robot was in contact with the ground. In contrast, in the floating scenario, errors were mainly observed only in coordinates with non-zero motion. The impedance controller maintained stable behavior across tasks, and the comparative analysis provided insights into how contact forces influence trajectory tracking.

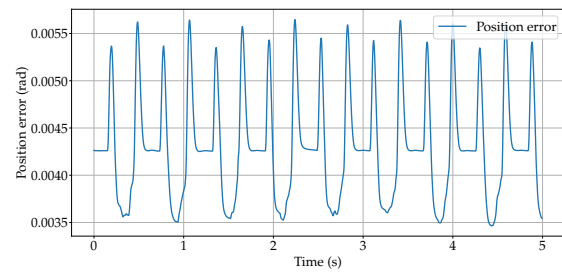
Future work may focus on extending the control strategy with dynamic modeling of the full robot, integrating ground contact detection, and adapting gait patterns in response to terrain conditions, which would further improve robustness and energy efficiency in real-world scenarios.

Appendix A

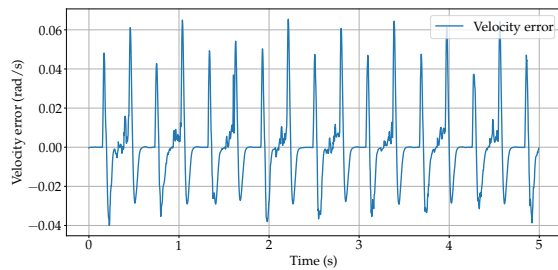
Time-domain trajectory following plots



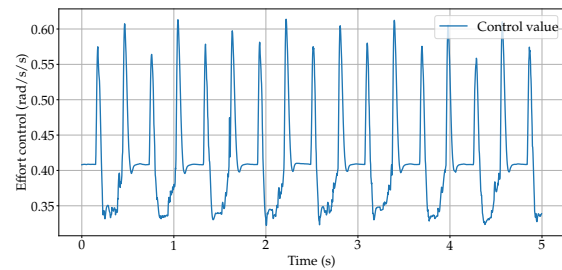
(a) Reference position and actual position



(b) Position error

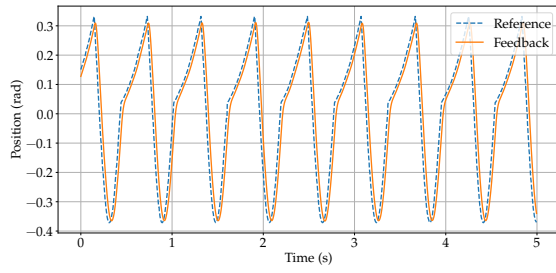


(c) Velocity error

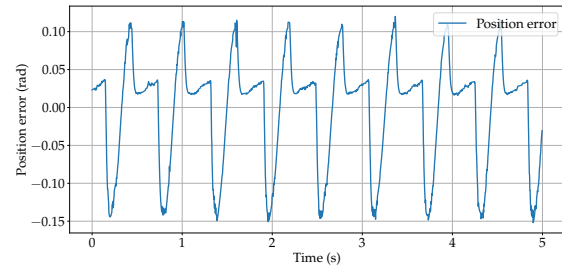


(d) Command values

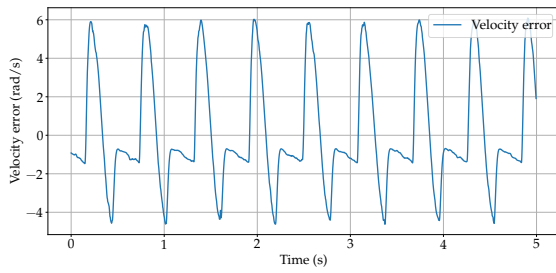
Figure A.1: Trajectory following of a q_0 joint.



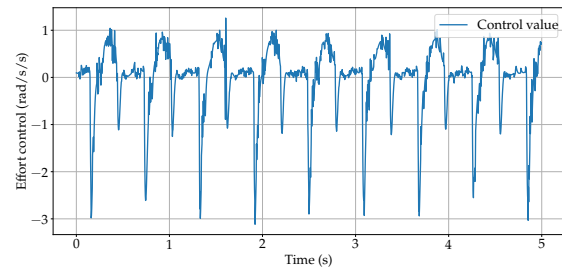
(a) Reference position and actual position



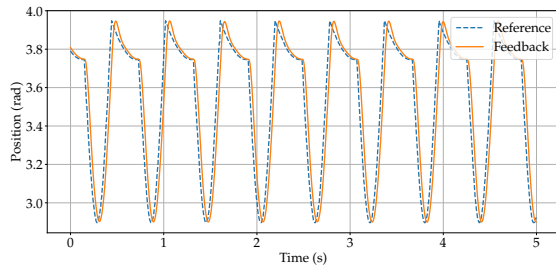
(b) Position error



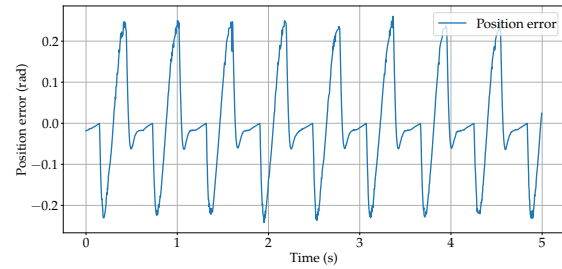
(c) Velocity error



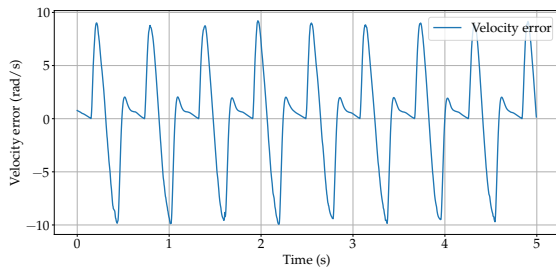
(d) Command values

Figure A.2: Trajectory following of a q_1 joint.

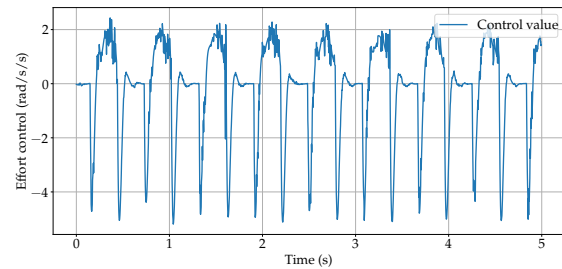
(a) Reference position and actual position



(b) Position error



(c) Velocity error



(d) Command values

Figure A.3: Trajectory following of a q_2 joint.

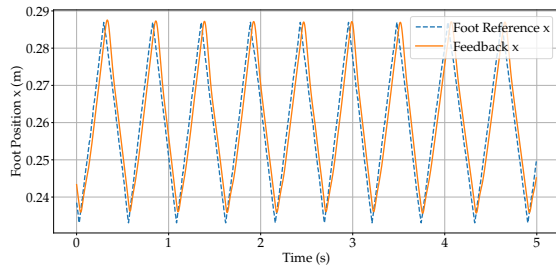
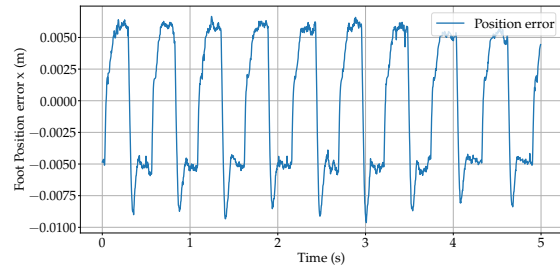
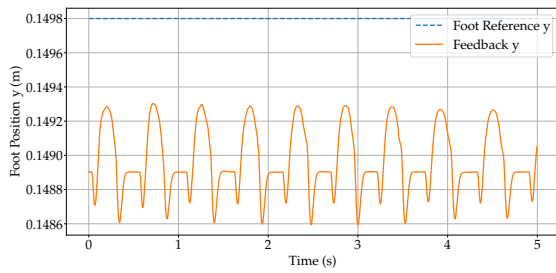
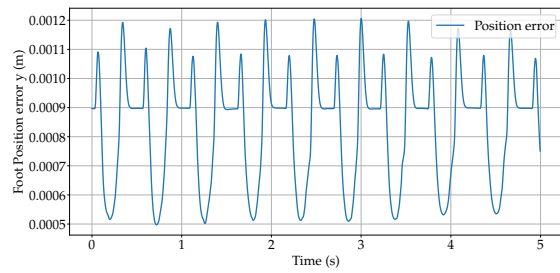
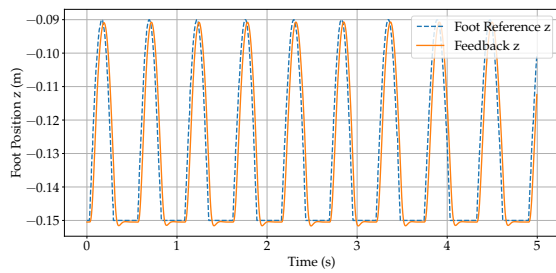
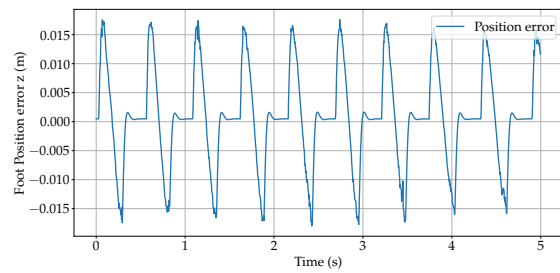
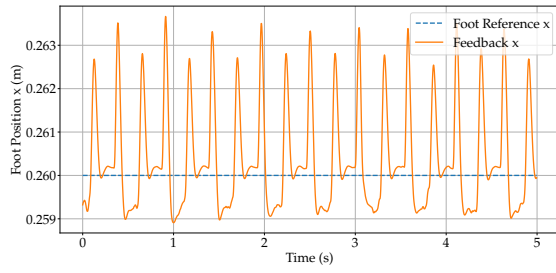
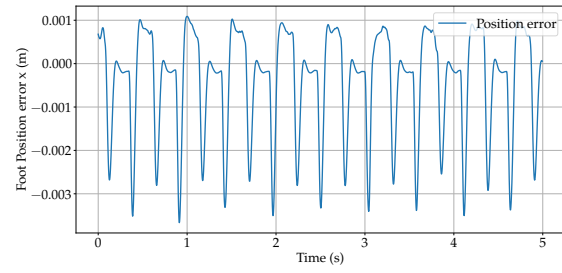
(a) Reference and actual position of foot x coordinate(b) Foot position error x coordinate(c) Reference and actual position of foot y coordinate(d) Foot position error y coordinate(e) Reference and actual position of foot z coordinate(f) Foot position error z coordinate

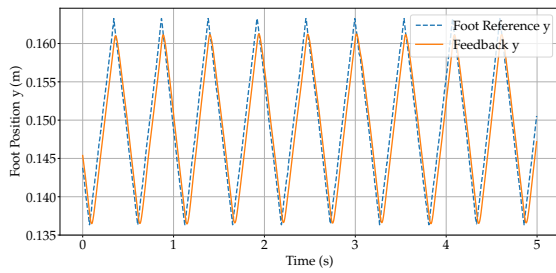
Figure A.4: Trajectory following of a position of foot in the task space while going forward (among X axis).



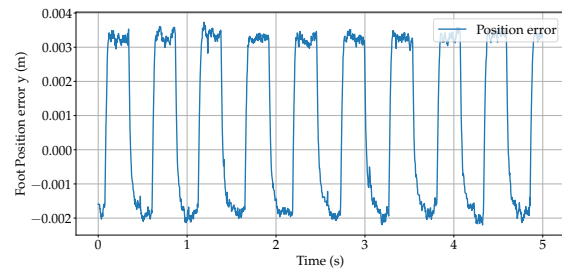
(a) Reference and actual position of foot x coordinate



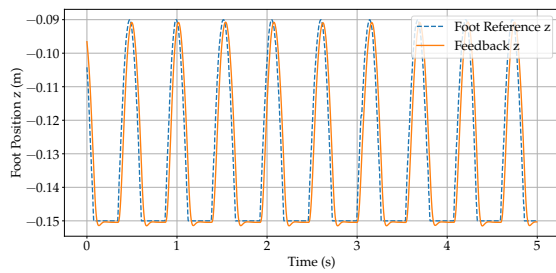
(b) Foot position error x coordinate



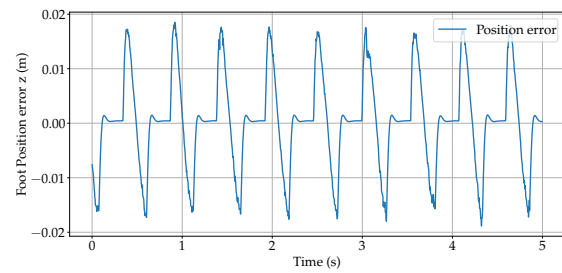
(c) Reference and actual position of foot y coordinate



(d) Foot position error y coordinate



(e) Reference and actual position of foot z coordinate



(f) Foot position error z coordinate

Figure A.5: Trajectory following of a position of foot in the task space while going sideways (among Y axis).

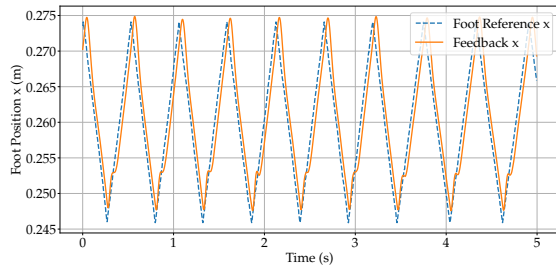
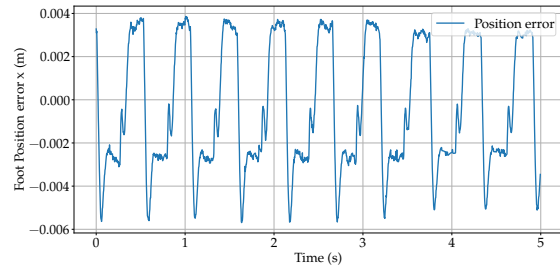
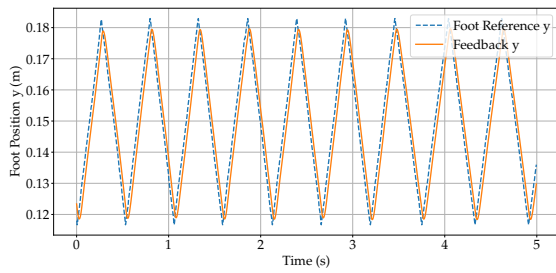
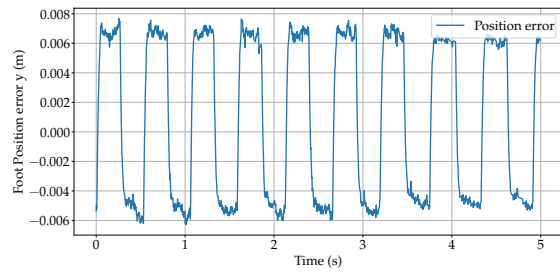
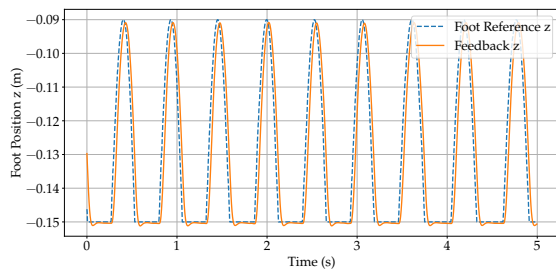
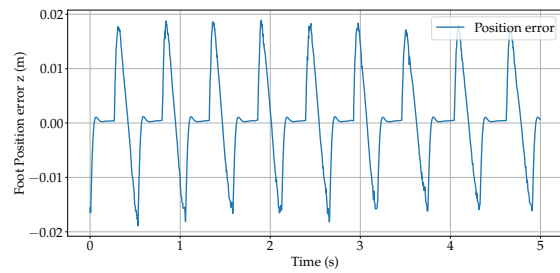
(a) Reference and actual position of foot x coordinate(b) Foot position error x coordinate(c) Reference and actual position of foot y coordinate(d) Foot position error y coordinate(e) Reference and actual position of foot z coordinate(f) Foot position error z coordinate

Figure A.6: Trajectory following of a position of foot in the task space while rotating clockwise.

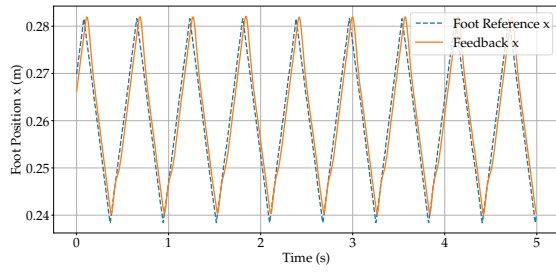
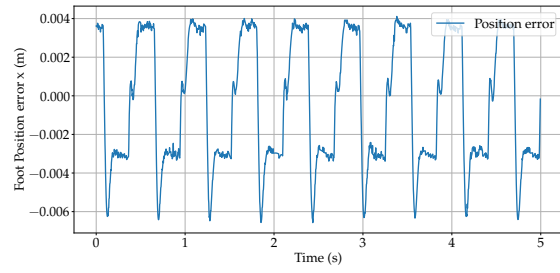
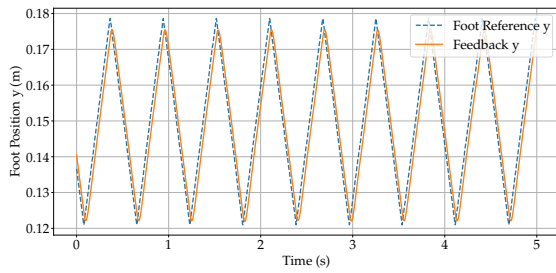
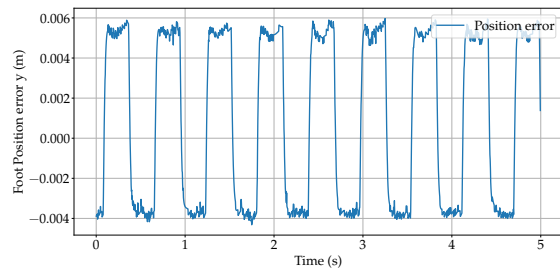
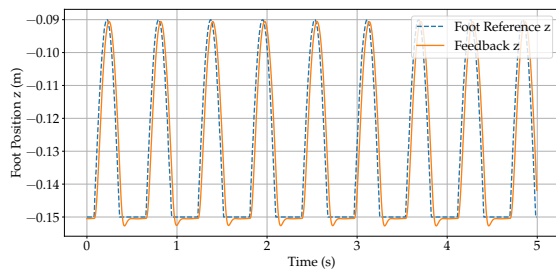
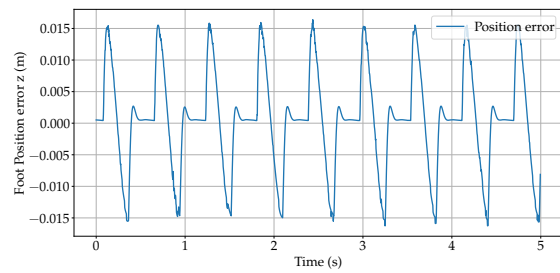
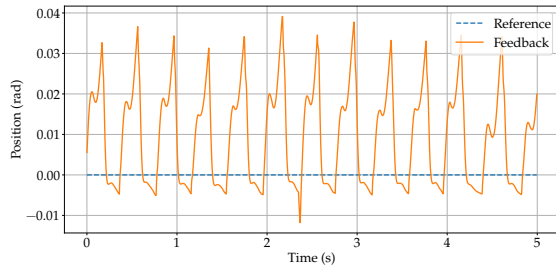
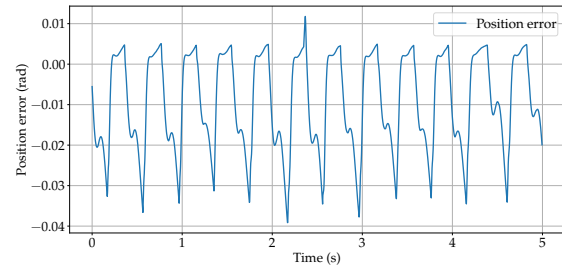
(a) Reference and actual position of foot x coordinate(b) Foot position error x coordinate(c) Reference and actual position of foot y coordinate(d) Foot position error y coordinate(e) Reference and actual position of foot z coordinate(f) Foot position error z coordinate

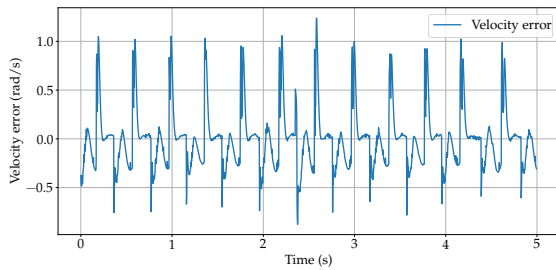
Figure A.7: Trajectory following of a position of foot in the task space while rotating counterclockwise.



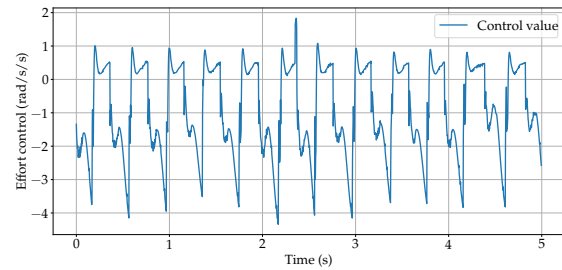
(a) Reference position and actual position



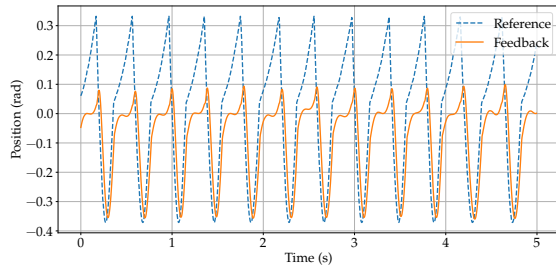
(b) Position error



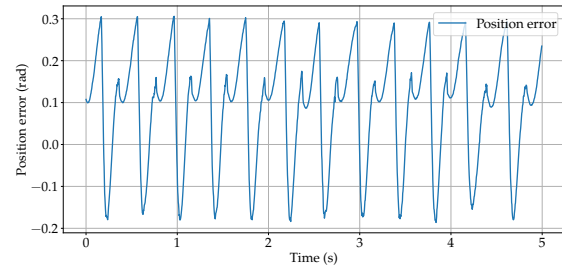
(c) Velocity error



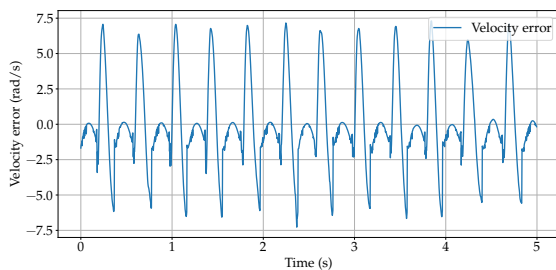
(d) Command values

Figure A.8: Trajectory following of a q_0 joint.

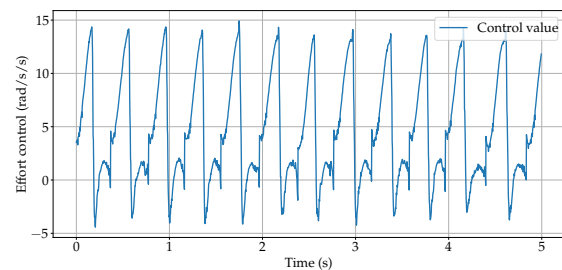
(a) Reference position and actual position



(b) Position error

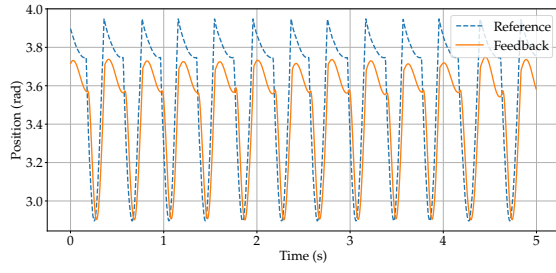


(c) Velocity error

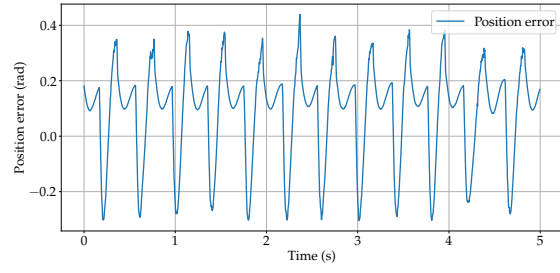


(d) Command values

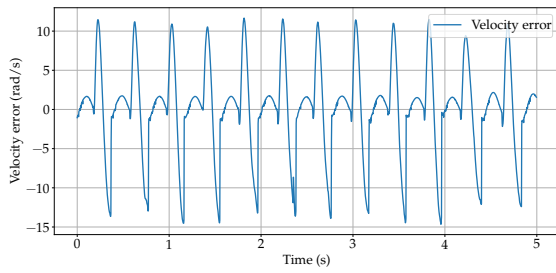
Figure A.9: Trajectory following of a q_1 joint.



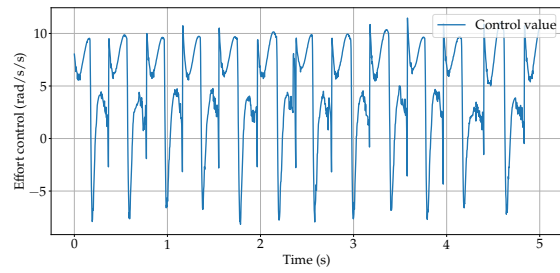
(a) Reference position and actual position



(b) Position error

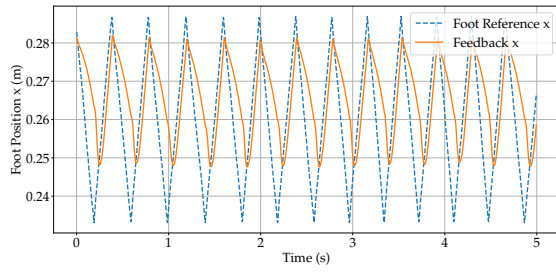


(c) Velocity error

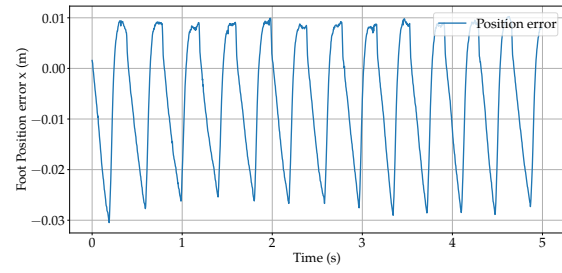


(d) Command values

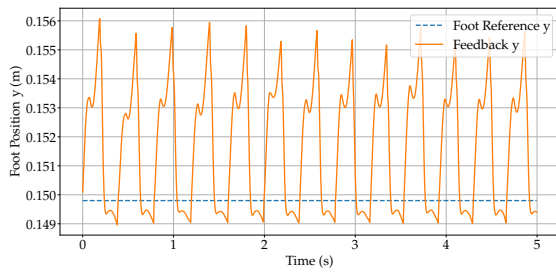
Figure A.10: Trajectory following of a q_2 joint.



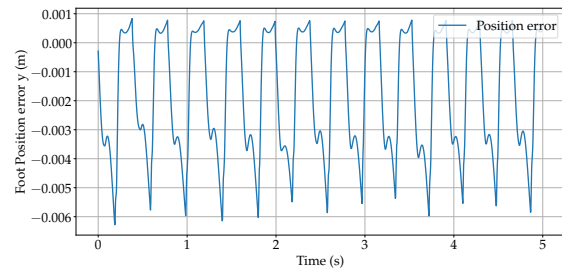
(a) Reference and actual position of foot x coordinate



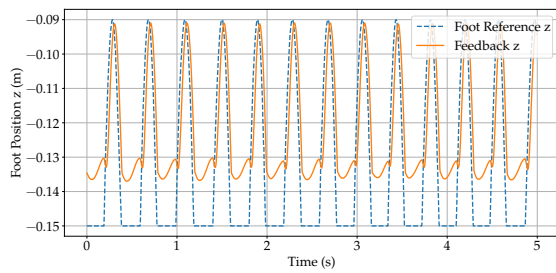
(b) Foot position error x coordinate



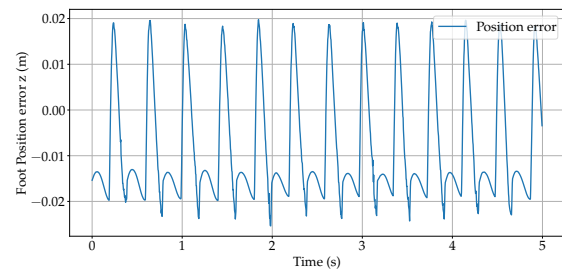
(c) Reference and actual position of foot y coordinate



(d) Foot position error y coordinate



(e) Reference and actual position of foot z coordinate



(f) Foot position error z coordinate

Figure A.11: Trajectory following of a position of foot in the task space while going on the ground forward (among X axis).

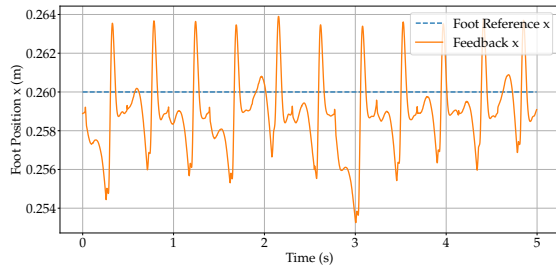
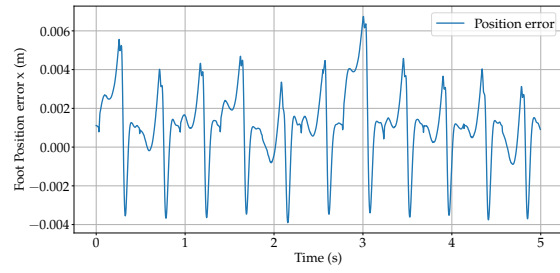
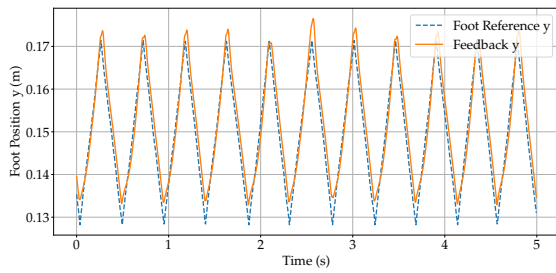
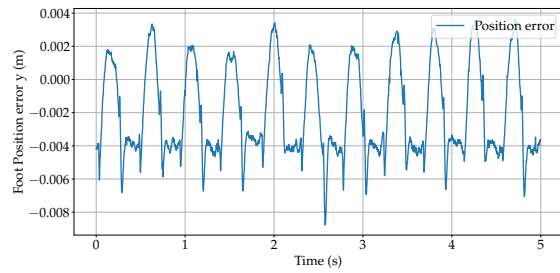
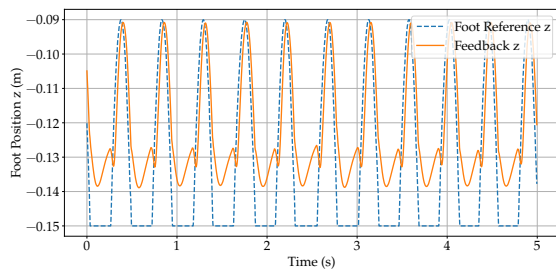
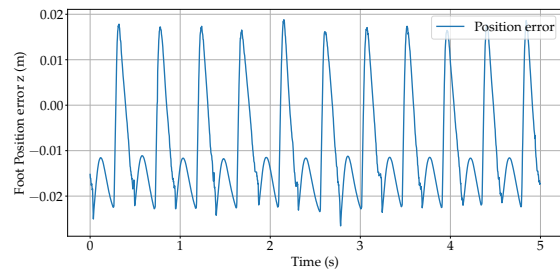
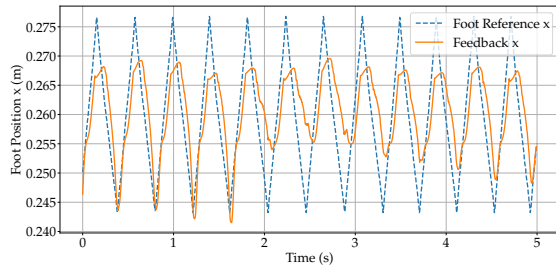
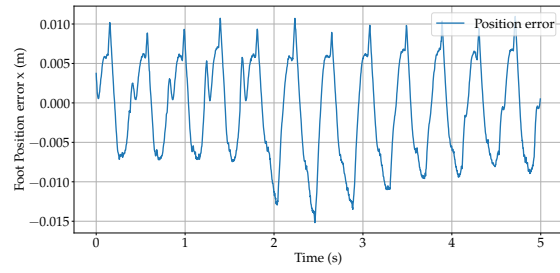
(a) Reference and actual position of foot x coordinate(b) Foot position error x coordinate(c) Reference and actual position of foot y coordinate(d) Foot position error y coordinate(e) Reference and actual position of foot z coordinate(f) Foot position error z coordinate

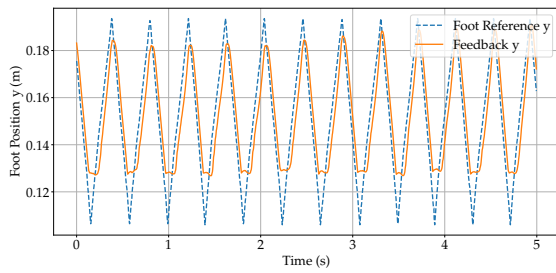
Figure A.12: Trajectory following of a position of foot in the task space while going on the ground sideways (among Y axis).



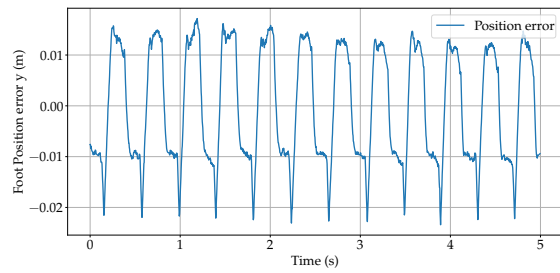
(a) Reference and actual position of foot x coordinate



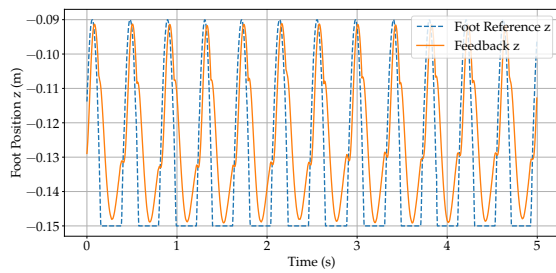
(b) Foot position error x coordinate



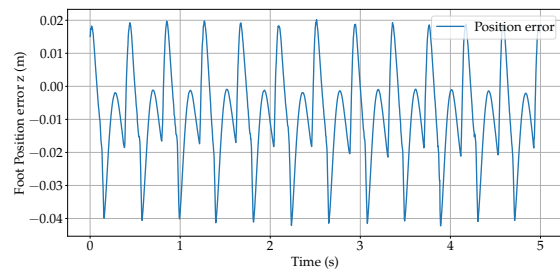
(c) Reference and actual position of foot y coordinate



(d) Foot position error y coordinate



(e) Reference and actual position of foot z coordinate



(f) Foot position error z coordinate

Figure A.13: Trajectory following of a position of foot in the task space while rotating on the ground clockwise.

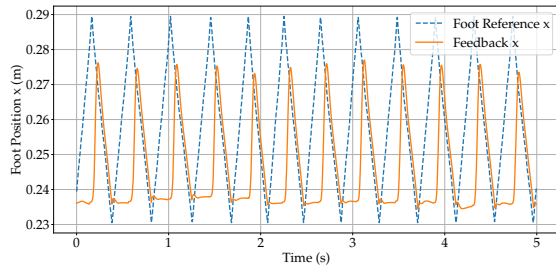
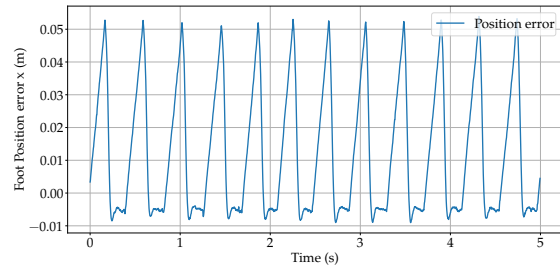
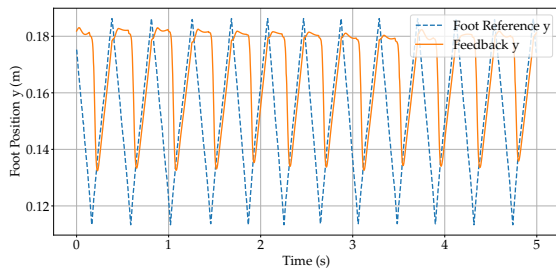
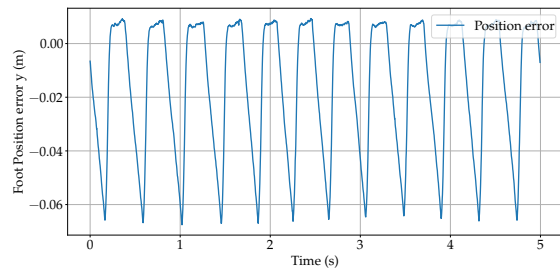
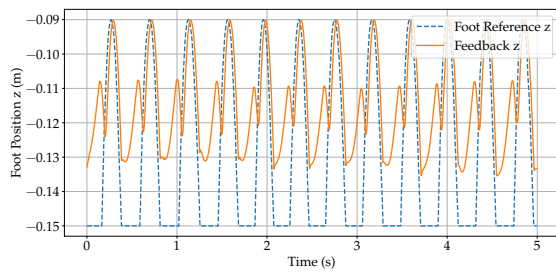
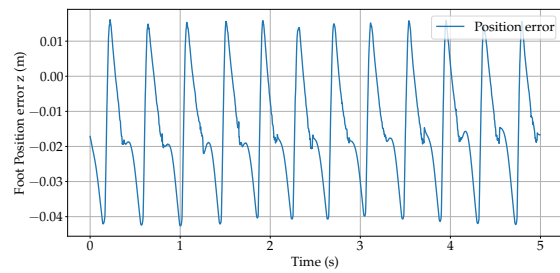
(a) Reference and actual position of foot x coordinate(b) Foot position error x coordinate(c) Reference and actual position of foot y coordinate(d) Foot position error y coordinate(e) Reference and actual position of foot z coordinate(f) Foot position error z coordinate

Figure A.14: Trajectory following of a position of foot in the task space while rotating the ground counterclockwise.

Bibliography

- [ALF⁺23] Michel Aractingi, Pierre-Alexandre Léziart, Thomas Flayols, Julien Perez, Tomi Silander, and Philippe Souères. Controlling the solo12 quadruped robot with deep reinforcement learning. *scientific Reports*, 13(1):11945, 2023.
- [ANYnd] ANYbotics. ANYmal D. <https://www.anybotics.com/anymal/>, n.d. Accessed: 2025-06-20.
- [Bosnd] Boston Dynamics. Spot. <https://www.bostondynamics.com/spot>, n.d. Accessed: 2025-06-20.
- [CC22] G. H. Cohen and G. A. Coon. Theoretical consideration of retarded control. *Transactions of the American Society of Mechanical Engineers*, 75(5):827–834, 07 2022.
- [DCWK⁺18] Jared Di Carlo, Patrick M. Wensing, Benjamin Katz, Gerardo Bleddt, and Sangbae Kim. Dynamic locomotion in the mit cheetah 3 through convex model-predictive control. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–9, 2018.
- [Dep25] Department of Mechanics and Machine Design, Wrocław University of Science and Technology. Laboratory of mechanisms and mobile robots (tmm), 2025. Accessed: 2025-06-22.
- [DFRnd] DFRobot. Bmx160 9-axis sensor module - sku: Sen0373. https://wiki.dfrobot.com/BMX160_9_Axis_Sensor_Module_SKU_SEN0373, n.d. Accessed: 2025-06-22.
- [Foc13] Michele Focchi. *Strategies To Improve the Impedance Control Performance of a Quadruped Robot*. Ph.D. thesis, University of Genoa and Istituto Italiano di Tecnologia (IIT), Genoa, Italy, April 2013.
- [FPW⁺24] Yanan Fan, Zhongcai Pei, Chen Wang, Meng Li, Zhiyong Tang, and Qinghua Liu. A review of quadruped robots: structure, control, and autonomous motion. *Advanced Intelligent Systems*, 6(6):2300783, 2024.
- [GKY18] Kevin G Gim, Joohyung Kim, and Katsu Yamane. Design of a serial-parallel hybrid leg for a humanoid robot. In *2018 IEEE International Conference on robotics and automation (ICRA)*, pages 6076–6081. IEEE, 2018.
- [GPM89] Carlos E Garcia, David M Prett, and Manfred Morari. Model predictive control: Theory and practice—a survey. *Automatica*, 25(3):335–348, 1989.

- [HRM⁺25] Abderrachid Hamrani, Md Munim Rayhan, Telusma Mackenson, Dwayne McDaniel, and Leonel Lagos. Smart quadruped robotics: a systematic review of design, control, sensing and perception. *Advanced Robotics*, 39(1):3–29, 2025.
- [KT15] Jackie Kay and Adolfo Rodriguez Tsouroukdissian. Real-time control in ROS and ROS 2.0. *ROSCon15*, 2015.
- [lie77] Automatic supervisory control of the configuration and behavior of multi-body mechanisms. *IEEE Transactions on Systems, Man, and Cybernetics*, 7(12):868–871, 1977.
- [MAB25] MAB Robotics. Md80 v3.0 motor controller. <https://www.mabrobotics.pl/product-page/md80-motor-controller>, 2025. High-performance brushless motor controller for robotics, supports CAN-FD, USB, position/velocity/torque control, and ROS integration.
- [MF68] Robert B McGhee and Andrew A Frank. On the stability properties of quadruped creeping gaits. *Mathematical Biosciences*, 3:331–351, 1968.
- [MFG⁺22] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.
- [MyA25] MyActuator. X-series actuators – downloads and documentation. <https://www.myactuator.com/downloads-xseries>, 2025. Accessed: 2025-06-12.
- [MYMO00] Mitsuharu Morisawa, Takahiro Yakoh, Toshiyuki Murakami, and Kouhei Ohnishi. A comparison study between parallel and serial linked structures in biped robot system. In *2000 26th Annual Conference of the IEEE Industrial Electronics Society. IECON 2000. 2000 IEEE International Conference on Industrial Electronics, Control and Instrumentation. 21st Century Technologies*, volume 4, pages 2614–2619. IEEE, 2000.
- [Mys12] D.H. Myszka. *Machines and Mechanisms: Applied Kinematic Analysis*. Pearson Prentice Hall, 2012.
- [Rai86] Marc H Raibert. Legged robots. *Communications of the ACM*, 29(6):499–514, 1986.
- [RBCG17] Hamed Razavi, Anthony M Bloch, Christine Chevallereau, and Jessy W Grizzle. Symmetry in legged locomotion: a new method for designing stable periodic gaits. *Autonomous Robots*, 41:1119–1142, 2017.
- [Rob23] PAL Robotics. Solo 12 advanced quadruped platform. <https://solo.pal-robotics.com/solo>, 2023. Accessed: 2025-06-20.
- [Rob24] MAB Robotics. CANDLE: Lightweight hardware interface for quadruped robots. <https://github.com/mabrobotics/candle>, 2024. Accessed: 2025-06-22.
- [ROS25] ROS 2. Getting started with ros2_control. https://control.ros.org/rolling/doc/getting_started/getting_started.html, 2025. Accessed: 2025-06-25.

- [Sem10] Claudio Semini. HyQ-design and development of a hydraulically actuated quadruped robot. *Ph. D. thesis, University of Genoa, Italy*, 2010.
- [TET12] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.
- [TM21] Krzysztof Tchoń and Robert Muszyński. *Analytic Mechanics*. Wrocław, Poland, 2021. Lecture Notes in Automation and Robotics.
- [TRR17] Dariusz Tomaszewski, Jacek Rapinski, and Pelc-Mieczkowska Renata. Concept of ahrs algorithm designed for platform independent imu attitude alignment. *Reports on Geodesy and Geoinformatics*, 104, 12 2017.
- [Tza13] Spyros G Tzafestas. *Introduction to mobile robot control*. Elsevier, 2013.
- [XHC⁺22] Rui Wu Xu, Kai Chin Hsieh, Un Hou Chan, Hoi Un Cheang, Wei Kai Shi, and Chi Tin Hon. Analytical review on developing progress of the quadruped robot industry and gaits research. In *2022 8th International Conference on Automation, Robotics and Applications (ICARA)*, pages 1–8. IEEE, 2022.
- [Yed03] Padmaraja Yedamale. Brushless DC (BLDC) motor fundamentals. *Microchip Technology Inc*, 20(1):3–15, 2003.